



Arora V 设计物理约束 用户指南

SUG1018-1.7, 2025-08-29

版权所有 © 2025 广东高云半导体科技股份有限公司

GOWIN高云、、Gowin、晨熙、云源以及高云均为广东高云半导体科技股份有限公司注册商标，本手册中提到的其他任何商标，其所有权利属其拥有者所有。未经本公司书面许可，任何单位和个人都不得擅自摘抄、复制、翻译本文档内容的部分或全部，并不得以任何形式传播。

免责声明

本文档并未授予任何知识产权的许可，并未以明示或暗示，或以禁止反言或其它方式授予任何知识产权许可。除高云半导体在其产品的销售条款和条件中声明的责任之外，高云半导体概不承担任何法律或非法律责任。高云半导体对高云半导体产品的销售和 / 或使用不作任何明示或暗示的担保，包括对产品的特定用途适用性、适销性或对任何专利权、版权或其它知识产权的侵权责任等，均不作担保。高云半导体对文档中包含的文字、图片及其它内容的准确性和完整性不承担任何法律或非法律责任，高云半导体保留修改文档中任何内容的权利，恕不另行通知。高云半导体不承诺对这些文档进行适时的更新。

版本信息

日期	版本	说明
2023/04/20	1.0	初始版本。
2023/06/30	1.1	新增 JTAGSEL_N Net、JTAG Net、SSPI Net、MODE Net、I2C Net 约束。
2023/08/18	1.2	● 更新 JTAG Net、SSPI Net、MODE Net、I2C Net 约束描述； ● 新增 MIPI_DPHY_RX 原语的约束示例； ● 删除 Timing Paths 功能。
2023/11/30	1.2.1	● 更新第 3 章 FloorPlanner 第 4 章 FloorPlanner 使用中的部分截图； ● 更新 “HCLK/GCLK Primitive Constraints” 选择 “Instance” 时弹出的对话框标题描述。
2024/02/02	1.2.2	● 新增 GW5A(T)-60 相关全局时钟约束和高速时钟约束描述； ● 更新图 3-22 反标 Port 布局信息。
2024/05/09	1.3	● 新增 IO 属性 Ctle 的描述； ● 新增 RECONFIG_N Net 约束； ● 新增 Primitive Group 约束支持通配符的描述。
2024/06/28	1.4	● 更新图 3-21 Back-annotate Physical Constraints 对话框； ● 新增 GW5A(N)®T-15 相关全局时钟约束和高速时钟约束描述； ● 新增 IO 属性 Slew Rate 的描述。
2024/08/09	1.5	● 新增 GW5A(®)(R)T-15 的 DSPLite、SCRAMBLE 原语相关描述； ● 新增 IO 属性 Pull Strength 的描述； ● 更新 IO 属性 Vref 的描述； ● 新增 Reset Layout 选项的描述； ● 更新图 3-23 View 菜单和图 3-24 Windows 菜单； ● 新增 3.2 启动中记忆界面布局功能的相关描述； ● 更新 2.10.1 ADC 输入电压源约束的部分描述。
2025/06/27	1.6	更新第 2 章物理约束语法规则描述。
2025/08/29	1.7	新增 2.10.8 NET_NO_REP 约束章节。

目录

目录	i
图目录	iii
表目录	vi
1 关于本手册	1
1.1 手册内容	1
1.2 相关文档	1
1.3 术语、缩略语	2
1.4 技术支持与反馈	2
2 物理约束语法规则	3
2.1 I/O 位置约束	3
2.2 I/O 属性约束	4
2.3 原语位置约束	5
2.4 组约束	8
2.4.1 原语组约束	9
2.4.2 相对组约束	10
2.5 资源预留约束	11
2.6 参考电压约束	12
2.7 全局时钟分配约束	13
2.8 全局时钟原语约束	14
2.9 高速时钟原语约束	15
2.10 其他约束	16
2.10.1 ADC 输入电压源约束	16
2.10.2 JTAGSEL_N Net 约束	17
2.10.3 RECONFIG_N Net 约束	18
2.10.4 JTAG Net 约束	18
2.10.5 SSPI Net 约束	19
2.10.6 MODE Net 约束	20
2.10.7 I2C Net 约束	21
2.10.8 NET_NO_REP 约束	22

3 FloorPlanner	23
3.1 简介	23
3.2 启动	24
3.3 界面	25
3.3.1 菜单栏	26
3.3.2 Summary 和 Netlist 窗口	36
3.3.3 Package View 窗口	39
3.3.4 Chip Array 窗口	43
3.3.5 Constraint 编辑窗口	49
3.3.6 Message 窗口	54
4 FloorPlanner 使用	55
4.1 新建约束文件	55
4.2 编辑约束文件	57
4.2.1 编辑约束示例	57
4.2.2 编辑 I/O 约束	59
4.2.3 编辑原语约束	60
4.2.4 编辑组约束	61
4.2.5 编辑资源预留约束	65
4.2.6 编辑全局时钟分配约束	66
4.2.7 编辑全局时钟约束	67
4.2.8 编辑高速时钟约束	68
4.2.9 编辑参考电压约束	69

图目录

图 3-1 菜单栏启动 FloorPlanner	24
图 3-2 Process 窗口启动	24
图 3-3 Start Page 窗口启动	25
图 3-4 FloorPlanner 界面.....	26
图 3-5 File 菜单.....	26
图 3-6 Open Physical Constraints	27
图 3-7 Constraints 菜单	27
图 3-8 原语查找对话框	28
图 3-9 新建原语组对话框.....	29
图 3-10 正确原语组对话框.....	29
图 3-11 无效位置	30
图 3-12 无效位置	30
图 3-13 创建相对位置组对话框	31
图 3-14 正确的相对组对话框	31
图 3-15 预留约束	31
图 3-16 时钟约束	32
图 3-17 创建全局时钟约束.....	33
图 3-18 创建高速时钟约束.....	33
图 3-19 参考电压约束.....	34
图 3-20 Tools 菜单	34
图 3-21 Back-annotate Physical Constraints 对话框.....	35
图 3-22 反标 Port 布局信息	35
图 3-23 View 菜单.....	35
图 3-24 Windows 菜单.....	36
图 3-25 Summary 窗口	36
图 3-26 Netlist 窗口	37
图 3-27 BUS 和非 BUS 结合显示	38
图 3-28 层级显示	38
图 3-29 Netlist 右键功能	39
图 3-30 GW5AT-138-FPBGA676A Package view 窗口	40

图 3-31 Package View 右键功能	41
图 3-32 差分对显示	41
图 3-33 Top View	42
图 3-34 Bottom View	42
图 3-35 Chip Array 窗口	43
图 3-36 网格模式约束	44
图 3-37 宏单元模式约束	44
图 3-38 原语模式约束	45
图 3-39 Chip Array 右键功能	47
图 3-40 Show Place View 显示	47
图 3-41 鼠标悬浮显示	48
图 3-42 右键高亮	49
图 3-43 I/O 约束窗口	51
图 3-44 原语约束窗口	51
图 3-45 组约束窗口	52
图 3-46 预留约束窗口	52
图 3-47 时钟约束窗口	53
图 3-48 全局时钟约束窗口	53
图 3-49 高速时钟约束窗口	54
图 3-50 Vref 约束窗口	54
图 3-51 Message 窗口	54
图 4-1 新建约束文件	55
图 4-2 选择器件	56
图 4-3 保存输出文件	57
图 4-4 拖拽到 Chip Array 创建 I/O Constraints	59
图 4-5 拖至 Package View 创建 I/O Constraints	60
图 4-6 拖拽到 Chip Array 创建 Primitive Constraints	61
图 4-7 Group Constraints 编辑器右键菜单	61
图 4-8 创建 Primitive Group Constraints	62
图 4-9 Primitive Group Constraints	63
图 4-10 Relative Group Constraints 创建	64
图 4-11 Relative Group Constraints	65
图 4-12 创建 Resource Reservation 约束	65
图 4-13 Resource Reservation	66
图 4-14 Clock Net Constraints 约束创建	67
图 4-15 Clock Net Constraints 约束	67
图 4-16 GCLK Primitive Constraints 创建	68
图 4-17 GCLK Primitive Constraints	68

图 4-18 HCLK Primitive Constraints 创建	69
图 4-19 HCLK Primitive Constraints.....	69
图 4-20 Vref Constraints 创建.....	69
图 4-21 拖拽至 Chip Array 窗口生成 Vref Constraints Location 信息.....	70
图 4-22 拖拽至 Package View 窗口生成 Vref Constraints location 信息	71
图 4-23 Vref Constraints 名字重复	72

表目录

表 1-1 术语、缩略语	2
表 2-1 DCS/DCE 可约束 Position.....	14
表 2-2 CLKDIV/DLLDLY 可约束 Position	16

1 关于本手册

1.1 手册内容

本手册主要描述高云半导体 FloorPlanner 的界面和使用以及 Arora V FPGA 产品物理约束语法规则，旨在帮助用户快速实现物理约束。因软件版本更新，部分信息可能会略有差异，具体以用户软件版本信息为准。

所有物理约束优先级都高于 Gowin 云源软件中布局布线全局设置的优先级。Gowin 云源软件可通过菜单栏“Project > Configuration > Place & Route”进行布局布线全局设置。

1.2 相关文档

通过登录高云半导体网站 www.gowinsemi.com.cn 可下载、查看以下相关文档：

- [DS981, GW5AT 系列 FPGA 产品数据手册](#)
- [DS1103, GW5A 系列 FPGA 产品数据手册](#)
- [DS1239, GW5AST 系列 FPGA 产品数据手册](#)
- [DS1105, GW5AS 系列 FPGA 产品数据手册](#)
- [DS1108, GW5AR 系列 FPGA 产品数据手册](#)
- [DS1118, GW5ART 系列 FPGA 产品数据手册](#)
- [SUG100, Gowin 云源软件用户指南](#)
- [UG306, Arora V 时钟资源\(Clock\)用户指南](#)
- [UG704, Arora V 138K FPGA 产品编程配置手册](#)
- [UG714, Arora V 25K FPGA 产品编程配置手册](#)
- [UG718, Arora V 60K FPGA 产品编程配置手册](#)
- [UG720, Arora V 15K FPGA 产品编程配置手册](#)

1.3 术语、缩略语

本手册中的相关术语、缩略语及相关释义请参见表 1-1。

表 1-1 术语、缩略语

术语、缩略语	全称	含义
BSRAM	Block SRAM	块状静态随机存储器
CFU	Configurable Function Unit	可配置功能单元
CLKDIV	Clock Divider	时钟分频器
CLS	Configurable Logic Section	可配置逻辑块
DCS	Dynamic Clock Selector	动态时钟选择器
DDRDLL	Double Data Rate Delay-locked Loop	双倍数据速率延时锁相环
DLLDLY	DLL Delay	DLL 延迟
DQS	Bidirectional Data Strobe Circuit for DDR Memory	DDR 存储器双向数据选通电路
FloorPlanner	FloorPlanner	物理约束编辑器
FPGA	Field Programmable Gate Array	现场可编程门阵列
GCLK	Global Clock	全局时钟
I/O	Input/Output	输入/输出
IDE	Integrated Development Environment	集成开发环境
LUT	Look-up Table	查找表
LW	Long Wire	时钟长线资源
PLL	Phase-locked Loop	锁相环
SSRAM	Shadow SRAM	分布式静态随机存储器
VREF	Voltage Reference	参考电压

1.4 技术支持与反馈

高云半导体提供全方位技术支持，在使用过程中如有任何疑问或建议，可直接与公司联系：

网址：www.gowinsemi.com.cn

E-mail：support@gowinsemi.com

Tel: +86 755 8262 0391

2 物理约束语法规范

2.1 I/O 位置约束

I/O 位置约束可将 `port` 约束到指定 IO 位置处。具体 IO 位置可参考对应器件的 [Pinout 手册](#)。

语法

```
IO_LOC "obj_name" obj_location [exclusive];
```

语法规范

- 约束语句以英文分号 “;” 结尾;
- 语句中约束对象 `obj_name` 需使用英文双引号引用;
- 语句中约束位置 `obj_location` 可指定多个, 多个位置之间需用英文逗号分隔, 如多个 IO 位置 “A11,B2”;
- 约束语句各部分以空格进行间隔。

约束元素

- `obj_name`

`obj_name` 为案例中 `port` 名称。

- `obj_location`

`obj_location` 为 IO 位置, 如 “A11”、“B12” 等, 若指定多个位置, 位置之间需要用英文逗号分隔, 如 “A11,B2”。

- `exclusive`

`exclusive` 为可选项, 在约束位置之后, 表明该约束语句中的 `obj_location` 仅可以放置 `obj_name` 指定的对象。

应用举例

示例 1

```
IO_LOC "port0" G2;
```

//对象 port0 被约束在 GW5AT-15B-MBGA132 封装 G2 位置。

示例 2

```
IO_LOC "port1" P10,N10,G2;
```

//对象 port1 被约束在 GW5AT-15B-MBGA132 封装 P10,N10,G2 的位置，布局时将取三个位置中第一个合理位置进行布局。

示例 3

```
IO_LOC "port2" P10,N10,G2 exclusive;
```

//在 GW5AT-15B-MBGA132 封装中，对象 port2 被约束在封装 P10,N10,G2，且 P10,N10,G2 三个位置仅可以被 port2 占用。

2.2 I/O 属性约束

I/O 属性约束，用于设定 I/O 的各种属性值。如 port 的电平标准 IO_TYPE，上拉/下拉模式 PULL_MODE，驱动能力 DRIVE 等，详细属性设置标准请参考各器件[数据手册](#)。

语法

```
IO_PORT " obj_name " attribute = attribute_value;
```

注!

- 一个约束语句中可设定多个属性，各个属性之间使用空格分隔。
- 语法规则可参考 2.1 章节中的[语法规范](#)。

约束元素

- obj_name

obj_name 为案例中 port 名称。

- attribute

IO 属性名称

- attribute value

IO 属性所对应的属性值

需要 IO 属性约束的 port 的属性和属性值。具体可约束属性以及属性值可参考文档 [UG304, Arora V 可编程通用管脚\(GPIO\)用户指南](#)。

应用举例

示例 1

```
IO_PORT "port_1" IO_TYPE = LVTTL33;
```

//设置 port_1 的 IO_TYPE 为 LVTTL33。

示例 2

```
IO_PORT "port_2" IO_TYPE = LVTTL33 PULL_MODE = KEEPER;
```

//设置 port_2 的 IO_TYPE 为 LVTTL33, PULL_MODE 属性值为 KEEPER。

2.3 原语位置约束

Primitive Constraints 用于将原语约束到指定位置，可以通过 Primitive Constraints 对 LUT、DFF、DL、MUX、IOLOGIC、BSRAM、SSRAM、DSP、DDRDL、PLL、DQS、MIPI_DPHY_RX、DSPLite、SCRAMBLE 等进行约束。

语法

```
INS_LOC "obj_name" obj_location [exclusive];
```

注!

语法规则可参考 2.1 章节中的[语法规则](#)。

约束元素

- obj_name

约束对象，原语的例化名称。

- exclusive

关键字“exclusive”为可选项，表明该约束语句中的 obj_location 仅可以放置 obj_name 指定的对象。

- obj_location

obj_location 为 obj_name 指定原语的合理位置，obj_location 包含如下几类：

1. LUT 约束位置

- 单一位置信息，指定到 LUT，如：RxCy[0-3][A-B];
- 位置信息为一个范围，指定多行或多列。
 - 包含多个 CLS 或 LUT：“RxCm”、“RxCm[0-3]”;

- 指定多行：“R[x:y]Cm”、“R[x:y]Cm[0-3]”、“R[x:y]Cm[0-3][A-B]”；
- 指定多列：“RxC[m:n]”、“RxC[m:n][0-3]”、“RxC[m:n][0-3][A-B]”；
- 指定多行多列：“R[x:y]C[m:n]”、“R[x:y]C[m:n][0-3]”、“R[x:y]C[m:n][0-3][A-B]”。

注!

- LUT 位置信息中的 x、y 指的是 GRID 的行信息；
- LUT 位置信息中的 m、n 指的是 GRID 的列信息；
- LUT 位置信息中的 R 代表 GRID 的行，C 代表 GRID 的列；
- LUT 位置信息中的 0-3 表示一个 GRID 某个 CLS 的编号；
- LUT 位置信息中的 A-B 表示一个 CLS 位置中的具体 LUT 位置的编号；
- 对于 MUX/DFF/DL 类型原语的约束位置与 LUT 相同。

2. PLL 约束位置

对于 PLL 约束位置书写格式为“PLL_L”或“PLL_R”，若左边可放置多个 PLL，可设为“PLL_L[0]”、“PLL_L[1]”...，若右边可放置多个 PLL，可设为“PLL_R[0]”、“PLL_R[1]”...

3. BSRAM 约束位置

BSRAM 约束位置书写格式为“BSRAM_R10[0]”（第 10 行第一个 BSRAM），“BSRAM_R10[1]”...

4. DSP 约束位置

DSP 约束位置书写格式为“DSP_R37[0]”（第 37 行第一个 DSP Block），“DSP_R37[1]”... 若需指定至 macro 时，可标记为：DSP_R37[0][A]或 DSP_R37[0][B]。

注!

一个 DSP Block 位置包括两个 macro，一个 macro 指的是一个 MULT12X12 可放置的位置。

5. DDRDLL 约束位置

DDRDLL 约束位置书写格式为“DDRDLLM_TL”，“DDRDLLM_TR”...

6. MIPI_DPHY_RX 约束位置

MIPI_DPHY_RX 约束位置书写格式为“QUAD[0]”，“QUAD[1]”。

注!

MIPI_DPHY_RX 约束支持器件：GW5A(S)(T)-138、GW5AT-75，不支持 exclusive。

7. DSPLite 约束位置

DSPLite 约束位置书写格式为“R19C26”，“R19C37”...，“R19C47”。

注！

DSPLite 约束支持器件：GW5A(N)(R)T-15。

8. SCRAMBLE 约束位置

SCRAMBLE 约束位置书写格式为“SCRM[0]”，“SCRM[1]”...

注！

SCRAMBLE 约束支持器件：GW5A(N)(R)T-15，不支持 exclusive 选项。

9. IOLOGIC 约束位置

IOLOGIC 约束位置为 IO 位置，具体 IO 位置可参考对应器件的 [Pinout 手册](#)。

应用举例

示例 1

```
INS_LOC "lut_1" R5C10[0][A];
```

//lut_1 被约束在 R5C10 的第 1 个 CLS 的第 1 个 LUT 的位置。

示例 2

```
INS_LOC "ins_2 " R5C6[2] exclusive;
```

//ins_2 被约束在 R5C6 的第 3 个 CLS 的位置，且该位置仅可以放置该约束对象。

示例 3

```
INS_LOC "ins_3" R[2:6]C2;
```

//ins_3 被约束在行坐标第二行到第六行，列坐标第二列的区域位置。

示例 4

```
INS_LOC "ins_4" R[2:4]C[2:6] exclusive;
```

//ins_4 被约束在行坐标为第二行到第四行，列坐标为第二列到第六列之间的区域位置，且该区域位置仅能被该约束对象所占用。

示例 5

```
INS_LOC "ins_5" R[2:4]C[2:6][1];
```

//ins_5 被约束在行坐标第二行到第四行，列坐标第二列到第六列之间的区域位置的任意一个 GRID 位置的第 2 个 CLS 中。

示例 6

```
INS_LOC "mipi_dphy_rx_name" QUAD[0];
```

//在 GW5AT-138B-PBGA484 封装中, 约束 mipi_dphy_rx_name 至第一个 MIPI_DPHY_RX 位置处。

示例 7

```
INS_LOC "scramble_name" SCRM[0];
```

//在 GW5AT-15B-MBGA132 封装中, 约束 scramble_name 至第一个 SCRAMBLE 位置处。

示例 8

```
INS_LOC "ides4_name" P10;
```

//在 GW5AT-15B-MBGA132 封装中, ides_name 被约束至 IO 的封装位置 P10。

示例 9

```
INS_LOC "pll_name" PLL_L[0];
```

//在 GW5AST-138B-PBGA484A 封装中, 约束 pll_name 至左边 PLL 的第一个位置。

示例 10

```
INS_LOC "bsram_name" BSRAM_R10[2];
```

//在 GW5AST-138B-PBGA484A 封装中, 约束 bsram_name 至第 10 行的第 3 个 BSRAM 位置处。

示例 11

```
INS_LOC "dsp_name" DSP_R37[2];
```

//在 GW5AST-138B-PBGA484A 封装中, 约束 dsp_name 位置第 37 行第 3 个 DSP Block。

示例 12

```
INS_LOC "ddrdll_name" DDRDLLM_TL;
```

//在 GW5AST-138B-PBGA484A 封装中, 约束 ddrdll_name 至顶部左边 DDRDLL 的位置。

2.4 组约束

Group Constraints 包括 Primitive Group Constraints 和 Relative Group Constraints, 如下所述。

2.4.1 原语组约束

Primitive Group 约束用于定义一个组约束，组是包含各类 Instance 对象的集合。通过 Primitive Group 约束，可将 Instance 如 LUT、DFF、DL、IOLOGIC、Buffer、BSRAM、SSRAM、DSP、PLL、DDRDLL、DQS、DSPLite 等类型添加到一个组中，并可通过约束该组的位置实现对该组中所有的对象的位置约束。同时，Primitive Group 约束支持通配符“?”和“*”的使用，“?”匹配一个字符，“*”可匹配零个或多个字符。

语法

Primitive Group 定义：

```
GROUP group_name = { "obj_names" } [exclusive];
```

```
GRP_LOC group_name group_location[exclusive];
```

注！

- 语法规范可参考 2.1 章节中的语法规范；
- Primitive Group 约束语法支持通配符“?”和“*”的使用，“?”匹配一个字符，“*”可匹配零个或多个字符；
- Primitive Group 约束语法包括组定义和组位置两句，缺一视为约束无效。

约束元素

- group_name

定义一个 Primitive Group 的名称。

- obj_name

obj_name 为添加至 Primitive Group 中的 Instance 对象。

- group_location

指定该组的约束位置，group_location 可取 IO、CFU、DQS、BSRAM、DSP、PLL、DDRDLL、DSPLite 的位置。

- exclusive

关键字“exclusive”为可选项，在组定义语句或组位置约束语句之后；

一个对象可以被多个组包含，但在组定义语句后添加“exclusive”关键字，表示该组内的对象仅可被该组所包含；

在位置约束语句之后使用“exclusive”，表示该约束位置仅可被该组内的对象所占用。

应用举例

示例 1

```
GROUP group_1 = { "lut1" "lut2" "lut3" "ins_4" };
```

```
GRP_LOC group_1 R[3:4]C4 exclusive;
```

//创建一个名为 **group_1** 的组，添加对象 **lut1**、**lut2**、**lut3**、**ins_4** 到该组中，组中所有对象可布局在区域 **R[3:4]C4** 的范围内，且该范围仅可布局 **group_1** 中的 **Instance** 对象。

实例 2

```
GROUP group_2 = { "lut_*" } exclusive;
```

```
GRP_LOC group_2 R[2:3]C[2:4];
```

//通配符 “*” 使用示例，创建一个名为 **group_2** 的组，会匹配添加所有以 “**lut_**” 开头的对象到该组中，且仅可属于该组，组中所有对象可布局在区域 **R[2:3]C[2:4]** 的范围内。

2.4.2 相对组约束

通过 **Relative Group Constraints**，可实现对 **instance** 如 **LUT1~LUT4**、**DFF**、**DL**、**MUX2** 对象的相对位置约束。

语法

Relative Group 约束定义：

```
REL_GROUP group_name = { "obj_names" };
```

```
INS_RLOC "obj_name" relative_location;
```

注！

- 语法规范可参考[语法规范](#)；
- **Relative Group** 约束语法包括组定义和组对象相对位置，缺一视为约束无效。

约束元素

- **obj_name**

定义一个 **Relative Group** 的名称。

- **relative_location**

组中各约束对象的相对位置信息描述。

应用举例

```
REL_GROUP grp_1 = { "ins_1" "ins_2" "ins_3" };
```

```
INS_RLOC "ins_1" R1C1;
```

```
INS_RLOC "ins_2" R2C3;
```

```
INS_RLOC "ins_3" R0C5;
```

//定义一个名为 `grp_1` 的相对组约束，并添加 `ins_1`、`ins_2`、`ins_3` 到 `grp_1` 中。以 `ins_1` 的相对位置 `R1C1` 为原点，`ins_2` 约束到相对 `R1C1` 到 `R2C3` 处，`ins_3` 约束到相对 `R1C1` 到 `R0C5` 处。

2.5 资源预留约束

通过 **Resource Reservation** 约束，可保留指定的位置或区域以避免在布局中使用。

语法

```
LOC_RESERVE location [ -res_obj ];
```

注！

语法规范可参考 2.1 章节中的[语法规范](#)。

约束元素

- `location`

预留的位置信息，只能预留 CFU、IO、BSRAM、DSP、DDRDL、PLL、DQS、MIPI_DPHY_RX、DSPLite、SCRAMBLE 类型的位置。

- `res_obj`

可选项，只对 `location` 中的 CFU 类型有效，有 LUT/REG 两个值可设，设置 LUT 代表只预留约束 `location` 中的 LUT 资源，设置 REG 代表只预留约束 `location` 中的 REG 资源。

应用举例

示例 1

```
LOC_RESERVE R2C3[0]-LUT;
```

//R2C3 第一个 CLS 中的 LUT 资源被预留。

举例 2

```
LOC_RESERVE R2C3[0]-REG;
```

//R2C3 第一个 CLS 中的 REG 资源被预留。

示例 3

```
LOC_RESERVE IOR3, IOR6, R2C3, R3C4;
```

//IOR3, IOR6, R2C3, R3C4 位置的资源在布局阶段被预留不能使用。

2.6 参考电压约束

芯片支持外部参考电压输入，对整个 BANK 有效。Vref Constraints 约束可用于对外部参考电压的输入管脚的名称和位置进行约束。

注！

- 可设置外部参考电压的输入管脚位置必须有 IOLOGIC 资源；
- Vref Constraints 和 Port 属性约束联合使用才有效。当 input 或 inout 类型的单端 Port, IO Type 为 SSTL/HSTL 时 Vref 属性可设置为创建的 Vref Constraints，表示该 Port 的参考电压使用 Vref Constraints 位置输入的外部参考电压。

语法

```
USE_VREF_DRIVER vref_name [location];
```

注！

语法规范可参考 2.1 章节中的语法规范。

约束元素

- vref_name

自定义的 VREF pin name

- location

任意含 IOLOGIC 资源的 I/O 位置可作为 VREF pin 约束的 location。

应用举例

示例 1

```
USE_VREF_DRIVER vref_pin L14;
```

```
IO_LOC "port_1" L13;
```

```
IO_PORT "port_1" IO_TYPE = SSTL18_I VREF=vref_pin;
```

//在 GW5AT-15B-MBGA132 中,定义一个名为“vref_pin”的 VREF pin, 设置 port_1 的 VREF 属性为 vref_pin, port_1 所约束位置 L13 与 L14 在同一 bank 上。

2.7 全局时钟分配约束

Clock Net Constraints 是对于设计中特定 net 走全局时钟线或不走时钟线的约束。

语法

```
CLOCK_LOC "net_name" global_clocks = signal_type;
```

注！

语法规则可参考 2.1 章节中的语法规则。

约束元素

- net_name

案例中的 net 名称。

- global_clocks

设置 net 要绕的时钟类型。

BUFG[0-15]: 绕具体一条全局时钟线;

BUFG: 绕全局时钟线;

LOCAL_CLOCK: 不绕时钟线。

- signal_type

设置 net 的信号类型。

CLK: signal_type 为时钟引脚的 net;

CE: signal_type 为时钟使能引脚的 net;

SR: signal_type 为 SET、RESET、CLEAR、PRESET 引脚的 net;

LOGIC: signal_type 为以上 signal_type 之外的 net;

指定多个 signal_type, 可使用 “|” 符号进行分隔。

注！

若 global_clocks 选择的是 LOCAL_CLOCK, 则 signal_type 不可选。

应用举例

示例 1

```
CLOCK_LOC "net" BUFG = CLK|CE;
```

//约束 CLOCK 对象 “net” 的 signal_type 为时钟引脚或时钟使能引脚的 net 走全局时钟线。

示例 2

```
CLOCK_LOC "net" LOCAL_CLOCK;
```

//约束 CLOCK 对象“net”不走时钟线。

示例 3

```
CLOCK_LOC "net" BUFG[0] = CLK;
```

//约束 CLOCK 对象“net”的 signal_type 为时钟引脚的 net 绕到芯片的第 1 条全局时钟线上。

2.8 全局时钟原语约束

GCLK Primitive Constraints 用于将 DCS、DCE 需要进行布局的全局时钟对象约束到指定的位置。

语法

```
INS_LOC "obj_name" position;
```

注！

语法规范可参考 2.1 章节中的[语法规范](#)。

约束元素

- obj_name

取 DCS、DCE 的实例化名称作为 obj_name。

- position

约束对象所放置的位置。

表 2-1 DCS/DCE 可约束 Position

器件	位置	
	DCE	DCS
GW5A(S)(T)-138/GW5AT-75	PTR0、PTR1、PTR2、PTR3、PTR0[0~5]、PTR1[0~5]、PTR2[0~5]、PTR3[0~5]、PBR0、PBR1、PBR2、PBR3、PBR0[0~5]、PBR1[0~5]、PBR2[0~5]、PBR3[0~5]、PG、SG、PG[0~11]、SG[0~15]	PTR0、PTR1、PTR2、PTR3、PTR0[0~1]、PTR1[0~1]、PTR2[0~1]、PTR3[0~1]、PBR0、PBR1、PBR2、PBR3、PBR0[0~1]、PBR1[0~1]、PBR2[0~1]、PBR3[0~1]、PG、PG[0~3]
GW5A(R)(S)-25	TOPLEFT、TOPRIGHT、	TOPLEFT、TOPRIGHT、

器件	位置	
	DCE	DCS
	BOTTOMLEFT、 BOTTOMRIGHT、STOP、 SBOTTOM	BOTTOMLEFT、 BOTTOMRIGHT
GW5A(T)- 60	PG、SG、PG[0~5]、 SG[0~7]	PG、PG[0~1]
GW5A(N)(R)T-15	PG、SG、PG[0~5]、 SG[0~7]	PG、PG[0~1]

注！

DCS/DCE 可约束 Position 的具体位置可参考文档：[UG306, Arora V 时钟资源\(Clock\) 用户指南](#)。

应用举例

```
INS_LOC "dcs_name" PTR0[1];

//在 GW5AT-138B-PBGA484A 封装中约束 DCS 对象 dcs_name 到
PTR0[1]位置。
```

2.9 高速时钟原语约束

通过 HCLK Primitive Constraints 约束，可将 CLKDIV、DLLDLY 约束到相关高速时钟位置。CLKDIV、DLLDLY 约束位置与普通 instance 对象约束位置不同，使用“BOTTOMSIDE”，“LEFTSIDE”，“RIGHTSIDE”表示约束位置的边。

语法

```
INS_LOC "obj_name" position;
```

注！

语法规范可参考 2.1 章节中的[语法规范](#)。

约束元素

- obj_name
取 CLKDIV、DLLDLY 的实例化名称作为 obj_name。
- position
约束对象所放置的位置。

表 2-2 CLKDIV/DLLDLY 可约束 Position

器件	位置	
	CLKDIV	DLLDLY
GW5A(S)(T)-138/GW5AT-75	BOTTOMSIDE[0~7]、 LEFTSIDE[0~7]、 RIGHTSIDE[0~7]	BOTTOMSIDE[0~3]、 LEFTSIDE[0~3]、 RIGHTSIDE[0~3]
GW5A(R)(S)-25	LEFTSIDE[0~3]、 RIGHTSIDE[0~3]、 BOTTOMSIDE[0~3]、 TOPSIDE[0~3]	BOTTOMSIDE[0~1]、 LEFTSIDE[0~1]、 RIGHTSIDE[0~1]、 TOPSIDE[0~1]
GW5A(T)- 60	LEFTSIDE[0~3]、 RIGHTSIDE[0~3]、 BOTTOMSIDE[0~7]、 TOPSIDE[0~3]	BOTTOMSIDE[0~3]、 LEFTSIDE[0~1]、 RIGHTSIDE[0~1]、 TOPSIDE[0~1]
GW5A(N)(R)T-15	LEFTSIDE[0~3]、 BOTTOMSIDE[0~3]	BOTTOMSIDE[0~1]、 LEFTSIDE[0~1]

应用举例

```
INS_LOC "clkdiv_name" LEFTSIDE[0];  
  
//在 GW5AT-138B-PBGA484A 封装中，将 clkdiv_name 约束布局到  
LEFTSIDE[0]处。
```

2.10 其他约束

2.10.1 ADC 输入电压源约束

ADC 输入电压源可以来自外部 IO，通过对 IO 位置进行 ADC 输入电压源约束来指定 ADC 提供输入电压源的入口。

注!

- ADC 输入电压约束支持器件：除 GW5A(S)(T)-138、GW5AT-75 器件外，其他器件都支持；
- 不同器件的 bus 对应 IO 位置可参考各器件的 pinout 手册。

语法

```
USE_ADC_SRC bus0~n location;  
  
注!
```

语法规范可参考 2.1 章节中的[语法规范](#)。

约束元素

- bus0~n

0~n 的可设值具体可参考各器件的 Pinout 手册中的“ADC_INPUT”列数据。

- location

location 为 IO 位置，location 形式只支持 Pinout 手册中“管脚名称”列的位置形式，且一次只能约束整个差分对位置，如 IOT48。

应用举例

USE_ADC_SRC bus2 IOT97

//在 GW5AT-60B-UBGA324A 封装中, 将 IO 位置 IOT97 作为 ADC 输入电压源的外部输入位置

2.10.2 JTAGSEL_N Net 约束

当使用 FPGA 内部逻辑控制 JTAGSEL_N 功能时，即在不断电第二次下载的时候，拉低 JTAGSEL_N 使得 JTAG 切换到配置下载功能，需要添加 JTAGSEL_N 的 net 物理约束。

语法

NET_LOC "obj_name" V_JTAGSELN;

注！

语法规范可参考 2.1 章节中的[语法规范](#)。

约束元素

- obj_name

内部逻辑的可绕线 net 作为 obj_name。

应用举例

NET_LOC "netname" V_JTAGSELN;

//将 netname 这条 net 来控制 JTAGSEL_N 的功能。

2.10.3 RECONFIG_N Net 约束

当使用 FPGA 内部逻辑控制 RECONFIG_N 的功能时，即在不断电第二次下载的时候，拉低 RECONFIG_N 使得 FPGA 切换到配置复位功能，需要添加 RECONFIG_N 的 net 物理约束。

语法

```
NET_LOC "obj_name" V_RECONFIGN;
```

注！

语法规范可参考 2.1 章节中的语法规范。

约束元素

- obj_name

内部逻辑的可绕线 net 作为 obj_name。

应用举例

```
NET_LOC "netname" V_RECONFIGN;
```

//将 netname 这条 net 来控制 RECONFIG_N 的功能。

2.10.4 JTAG Net 约束

JTAG 功能管脚包括 TCK、TMS、TDI、TDO。其中 TCK、TMS、TDI 功能可以通过 IO 专用配置实现，也可以通过 FPGA 内部逻辑控制专用配置实现，这两种实现方式是互斥的，且这些功能管脚只能选择相同的实现方式，即同时选择 IO 专用配置实现或者同时选择 FPGA 内部逻辑控制专用配置实现；另外，TDO 只能通过 IO 专用配置实现，且不受 TCK、TMS、TDI 的实现方式影响。

当使用 FPGA 内部逻辑控制 JTAG 的 TCK、TMS、TDI 功能时，需要添加 JTAG 的 net 物理约束。JTAG 的使用可具体参考 Arora V FPGA 产品编程配置手册。

- [UG704, Arora V 138K FPGA 产品编程配置手册](#)
- [UG714, Arora V 25K FPGA 产品编程配置手册](#)
- [UG718, Arora V 60K FPGA 产品编程配置手册](#)
- [UG720, Arora V 15K FPGA 产品编程配置手册](#)

语法

```
NET_LOC "obj_Name" V_TCK;
```

```
NET_LOC " obj_Name" V_TMS;
```

```
NET_LOC " obj_Name" V_TDI;
```

注！

语法规范可参考 2.1 章节中的语法规范。

约束元素

- obj_name

内部逻辑的可绕线 net 作为 obj_name。

应用举例

```
NET_LOC "netname0" V_TCK;
```

```
NET_LOC " netname1 " V_TMS;
```

```
NET_LOC " netname 2" V_TDI;
```

//将 netname0 这条 net 来控制 TCK 的功能，将 netname1 这条 net 来控制 TMS 的功能，将 netname2 这条 net 来控制 TDI 的功能。

2.10.5 SSPI Net 约束

SSPI 功能管脚包括 SI、SO、SSPI_WPN、CLKHOLD_N、SSPI_CLK、SSPI_CS_N，可以通过 IO 专用配置实现，也可以通过 FPGA 内部逻辑控制专用配置实现，两种实现方式是互斥的，且这些功能管脚只能选择相同的实现方式，即同时选择 IO 专用配置实现或者同时选择 FPGA 内部逻辑控制专用配置实现。

当使用 FPGA 内部逻辑控制 SSPI 的功能时，需要添加 SSPI 的 net 物理约束。SSPI 的使用可根据不同器件参考以下手册：

语法

```
NET_LOC " obj_Name" V_SSPI SI;
```

```
NET_LOC " obj_Name" V_SSPI SO;
```

```
NET_LOC " obj_Name" V_SSPI WPN;
```

```
NET_LOC " obj_Name" V_SSPI CLKHOLDN;
```

```
NET_LOC " obj_Name" V_SSPI CLK;
```

```
NET_LOC " obj_Name" V_SSPI CSN;
```

注！

语法规范可参考 2.1 章节中的语法规范。

约束元素

- obj_name

内部逻辑的可绕线 net 作为 obj_name。

应用举例

```
NET_LOC "netname0" V_SSPI SI;  
NET_LOC " netname1" V_SSPI SO;  
NET_LOC " netname2" V_SSPI WPN;  
NET_LOC "netname3" V_SSPI CLKHOLDN;  
NET_LOC " netname4" V_SSPI CLK;  
NET_LOC " netname5" V_SSPI CSN;
```

//将 netname0 这条 net 来控制 SI 的功能, 将 netname1 这条 net 来控制 SO 的功能, 将 netname2 这条 net 来控制 SSPI_WPN 的功能, 将 netname3 这条 net 来控制 CLKHOLD_N 的功能, 将 netname4 这条 net 来控制 SSPI_CLK 的功能, 将 netname5 这条 net 来控制 SSPI_CS_N 的功能。

2.10.6 MODE Net 约束

MODE 功能管脚包括 MODE0、MODE1、MODE2, 可以通过 IO 专用配置实现, 也可以通过 FPGA 内部逻辑控制专用配置实现, 两种实现方式是互斥的, 且这些功能管脚只能选择相同的实现方式, 即同时选择 IO 专用配置实现或者同时选择 FPGA 内部逻辑控制专用配置实现。

当使用 FPGA 内部逻辑控制 MODE 的功能时, 除了 MODE0、MODE1、MODE2 外, 还增加了 MODE_LD 信号, 该信号为加载信号, 在上升沿时加载内部逻辑控制的 MODE0~MODE2 的值, 即在 MODE_LD 为上升沿时切换 MODE0~MODE2 的值, 在第一个 MODE_LD 上升沿来之前使用的是下载码流时 MODE0~MODE2 设置的值。当使用 FPGA 内部逻辑控制 MODE 的功能时, 需要添加 MODE 的 net 物理约束。MODE 的使用可根据不同器件参考以下手册:

语法

```
NET_LOC " obj_Name" V_MODE0;  
NET_LOC " obj_Name" V_MODE1;  
NET_LOC " obj_Name" V_MODE2;  
NET_LOC "obj_Name" V_MODE_LD;
```

注!

语法规范可参考 2.1 章节中的[语法规范](#)。

约束元素

- obj_name

内部逻辑的可绕线 net 作为 obj_name。

应用举例

```
NET_LOC " netname0" V_MODE0;
```

```
NET_LOC " netname1" V_MODE1;
```

```
NET_LOC " netname2" V_MODE2;
```

```
NET_LOC " netname3" V_MODE_LD;
```

//将 netname0 这条 net 来控制 MODE0 的功能，将 netname1 这条 net 来控制 MODE1 的功能，将 netname2 这条 net 来控制 MODE2 的功能，将 netname3 这条 net 来控制 MODE_LD 的功能。

2.10.7 I2C Net 约束

I2C 功能管脚包括 SCL、SDA，可以通过 IO 专用配置实现，也可以通过 FPGA 内部逻辑控制专用配置实现，两种实现方式是互斥的，且这些功能管脚只能选择相同的实现方式，即同时选择 IO 专用配置实现或者同时选择 FPGA 内部逻辑控制专用配置实现。

当使用 FPGA 内部逻辑控制 I2C 的功能时，需要添加 I2C 的 net 物理约束。

语法

```
NET_LOC " obj_Name" V_SCL;
```

```
NET_LOC " obj_Name" V_SDA;
```

注！

语法规范可参考 2.1 章节中的[语法规范](#)。

约束元素

- obj_name

内部逻辑的可绕线 net 作为 obj_name。

应用举例

```
NET_LOC " netname0" V_SCL;
```

```
NET_LOC " netname1" V_SDA;
```

//将 netname0 这条 net 来控制 SCL 的功能，将 netname1 这条 net 来控制 SDA 的功能。

注!

I2C net 约束不支持器件：GW5A(S)(T)-138、GW5AT-75。

2.10.8 NET_NO_REP 约束

指定高扇出的 net 不进行资源复制的约束。

语法

```
NET_NO_REP "netName";
```

注!

语法规范可参考 2.1 章节中的[语法规范](#)。

约束元素

- netName

高扇出 net 作为 netName，可一次指定多个 netName。

应用举例

示例 1

```
NET_NO_REP "net1";
```

//指定高扇出 net，net1 不进行复制资源来优化扇出。

示例 2

```
NET_NO_REP "net2","net3";
```

//指定高扇出 net，net2 和 net3 不进行复制资源来优化扇出。

3 FloorPlanner

3.1 简介

FloorPlanner 是高云半导体面向市场自主研发的物理约束编辑器，支持对 I/O、Primitive（原语）、Group 等属性及位置信息的读取与编辑，同时可根据用户的配置生成新的布局与约束文件，文件中规定了 I/O 的属性信息，原语、模块的位置信息等。FloorPlanner 提供了简单快捷的布局与约束编辑功能，提高编写物理约束文件的效率。

FloorPlanner 功能特点：

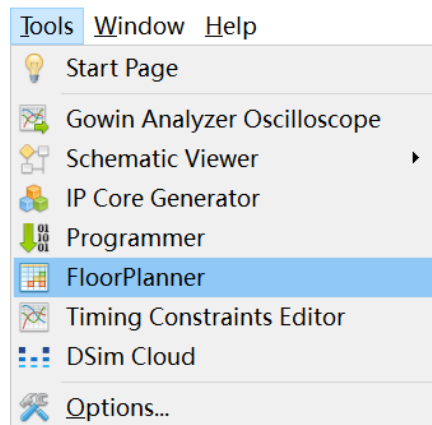
- 支持用户设计文件、约束文件的读入，约束文件的编辑，以及约束文件的输出
- 支持对用户设计文件中 IO Port、Primitive、Group 约束信息等的显示
- 支持用户新建、编辑、修改约束信息
- 支持 Chip Array 的网格模式、宏单元模式以及原语模式显示
- 支持依据 Package 信息的 Package View 显示
- 支持 Chip Array 和 Package View 的同步显示
- 支持约束位置信息的实时显示及区别显示
- 支持拖拽设置位置信息的功能
- 支持 IO Port 的属性配置功能，支持批量配置
- 支持 Clock Net Constraints 的显示、编辑功能
- 支持约束信息合法性检查的功能
- 支持 Back-annotate Physical Constraints 功能

3.2 启动

可通过以下三种方式启动 FloorPlanner:

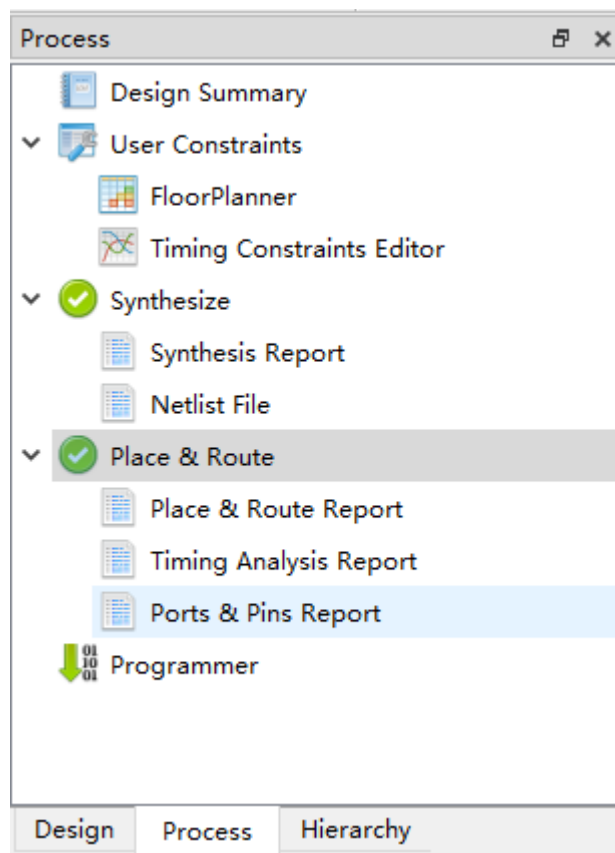
1. 单击“IDE > Tools”，打开“FloorPlanner”如图 3-1 所示；

图 3-1 菜单栏启动 FloorPlanner



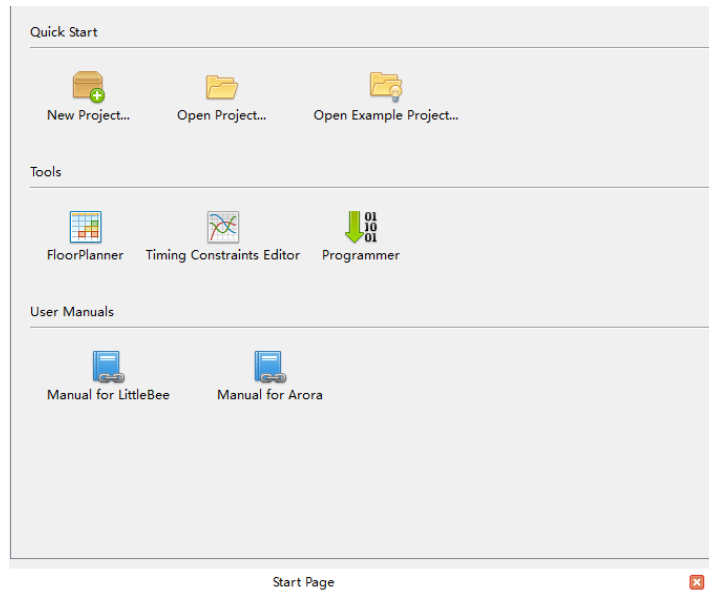
2. 建立工程在 Process 窗口运行 Synthesize 成功后, 双击“FloorPlanner”, 如图 3-2 所示。

图 3-2 Process 窗口启动



- 单击“IDE > Start Page > Tools > FloorPlanner”，打开“FloorPlanner”，如图 3-3 所示。

图 3-3 Start Page 窗口启动



注!

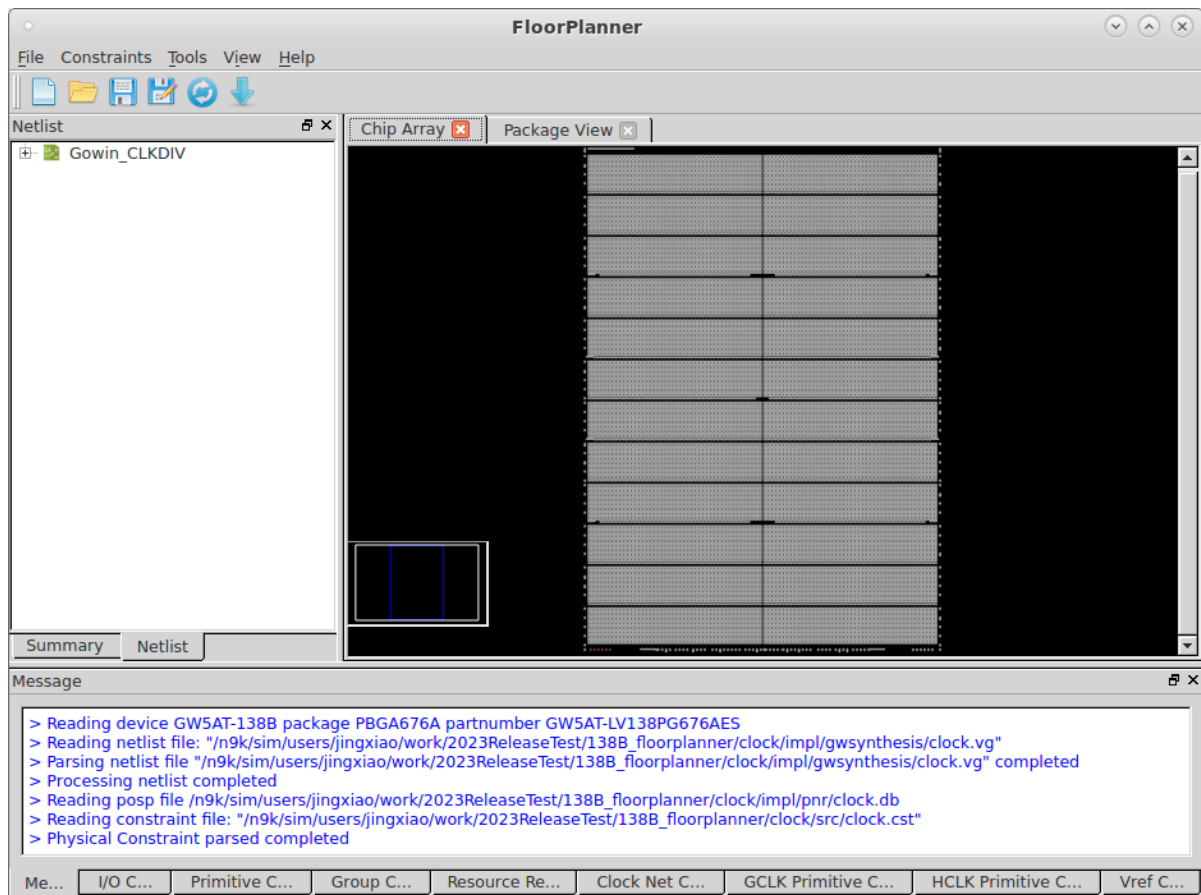
- 如需 FloorPlanner 进行约束，应先加载网表文件；
- 通过第一种和第二种方式启动 FloorPlanner 时，当前工程中的网表文件会自动加载；
- 通过第三种方式启动 FloorPlanner 时，需要通过“File > New”加载网表文件；
- 通过第一种和第二种方式启动 FloorPlanner 时，会记忆当前工程上一次关闭 Floorplanner 时的界面布局。

3.3 界面

新建或打开 FloorPlanner 界面（包含网表文件）如图 3-4 所示。

界面包括菜单栏、工具栏、Netlist 窗口、Summary 窗口、Chip Array 窗口、Package View 窗口、Message 窗口以及各类约束编辑窗口等。

图 3-4 FloorPlanner 界面



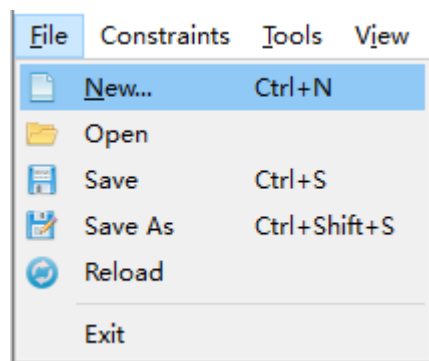
3.3.1 菜单栏

FloorPlanner 的菜单栏分为“File”、“Constraints”“Tools”、“View”及“Help”5个子菜单。

File 菜单

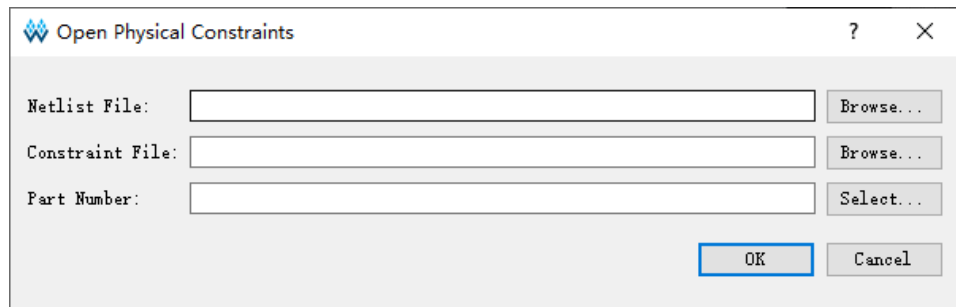
File 菜单如图 3-5 所示。

图 3-5 File 菜单



- **New:** 新建约束，添加用户设计，设置器件信息；
- **Open:** 打开约束，添加用户约束，设置器件信息如图 3-6；
- **Reload:** 当在磁盘或工程中对物理约束文件、布局文件进行修改后，可以重新加载；
- **Save:** 当前约束信息的修改信息覆盖原约束文件；
- **Save As:** 将当前约束信息的修改信息输出到用户指定的文件中，默认采用网表文件名作为约束文件名称，用户可修改；
- **Exit:** 退出 FloorPlanner。

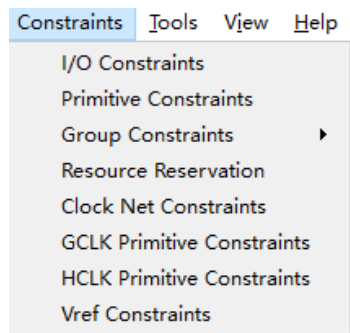
图 3-6 Open Physical Constraints



Constraints 菜单

Constraints 菜单栏如图 3-7 所示。

图 3-7 Constraints 菜单



Primitive Constraints

创建 Primitive 约束，选择 Primitive 创建对应的约束，右击选择 Select Primitive，可弹出如图 3-8 所示对话框。

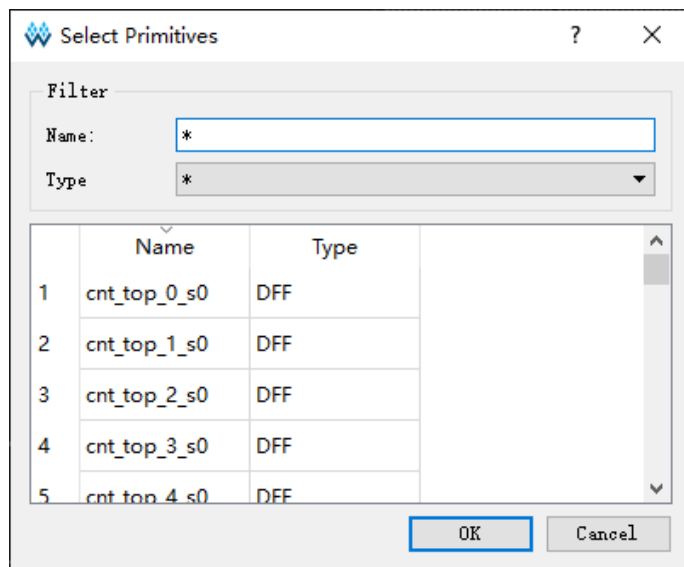
可通过 Primitives 名称或类型进行查找，选择对应的 Primitive；

1. 单击“OK”，产生约束信息，约束信息显示在主界面底部的“Primitive Constraints”约束编辑窗口中；
2. 用户可在编辑窗口中通过输入或拖拽的方式设置位置信息。

注！

所约束的位置在 Chip Array 窗口中呈浅蓝色高亮显示。

图 3-8 原语查找对话框



Group Constraints

Group Constraints, 包括 New Primitive Group 和 New Relative Group, 各功能介绍如下:

创建 Primitive Group。

1. 创建 Primitive Group 约束，右击选择 New Primitive Group，弹出如图 3-9 所示对话框；
2. 用户可设置 Group 的名称、包含的 Primitive 位置信息，以及 Group 的 Exclusive 信息；通过 “” 和 “” 两个按钮实现 Primitive 的添加和删除，正确创建的 Primitive Group 如图 3-10 所示；

注！

- Group 的名称、包含的 Primitive、Group 的位置为必填项；
- 可通过以下方式输入 Group 的位置信息：
 - 通过手动方式输入；
 - 建立 Group 约束前，在“Chip Array”窗口中，复制位置，粘贴到“New Primitive Group > Locations”中。
- 3. Primitive Group 创建配置完成后，单击“OK”，工具此时会对 Group 的位置信息进行语法检查。
 - 若位置信息不合理或不合法，会弹出如图 3-11 和如图 3-12 所示的提示框，用户需重新修改位置信息；

- 若无错误提示，单击“OK”，在 Chip Array 中会显示可用的位置。
4. 新产生的组约束显示在主界面底部的“Group Constraints”约束编辑窗口中，在“Group Constraints”约束编辑窗口中，双击 Primitive Group 约束可打开如图 3-10 所示的对话框，重新进行编辑修改。

图 3-9 新建原语组对话框

Group Name:

Members

Name	Type
------	------

☐ Exclusive

Locations

☐ Exclusive

图 3-10 正确原语组对话框

Group Name:

Members

Name	Type
cout_cZ	LUT4
out_Z[1]	DFFE

☐ Exclusive

Locations

☐ Exclusive

图 3-11 无效位置

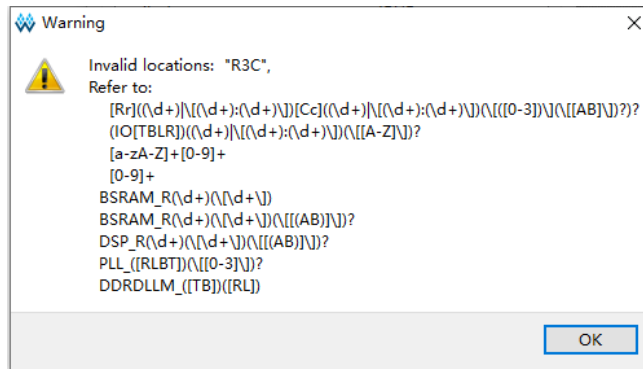
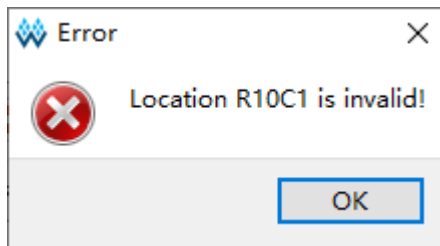


图 3-12 无效位置



创建 Relative Group

1. 创建 Relative Group 约束，右击选择“New Relative Group”，弹出如图 3-13 所示对话框；
2. 用户可设置 Group 的名称、包含的 Primitive 以及各 Primitive 对应的相对位置信息；可通过“+”和“-”实现 Primitive 的添加和删除，创建成功的 Relative Group 约束如图 3-14 所示；

注！

- Group 的名称、包含的 Primitive 及 Relative Location 为必填项；
 - 可通过以下方式输入 Group 的位置信息：
 - 通过手动方式输入；
 - 在建立 Group 约束前，在“Chip Array”窗口中，复制位置，粘贴到“New Relative Group > Relative Location”中。
3. 配置完成后单击“OK”，产生约束信息。
 4. 产生的约束信息显示在主界面底部的“Group Constraints”约束编辑窗口中；在编辑窗口中，双击约束，重新打开如图 3-14 所示的对话框，可进行编辑修改。

图 3-13 创建相对位置组对话框

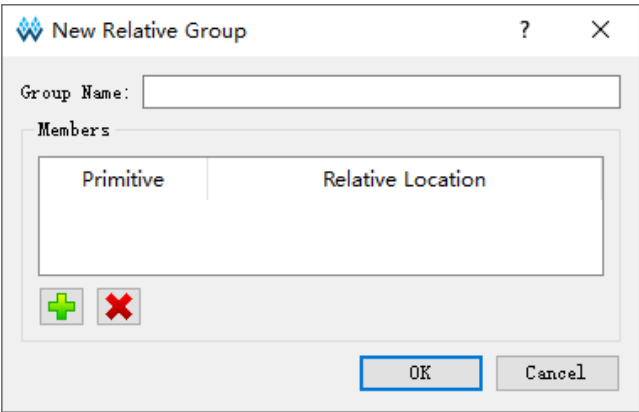
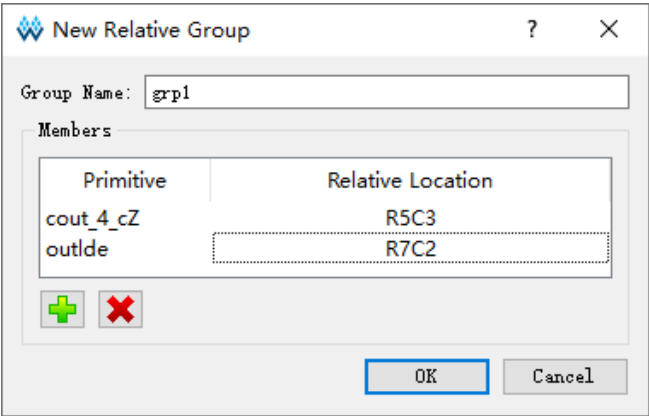


图 3-14 正确的相对组对话框



Resource Reservation

- 1. 创建 Resource Reservation 约束，在主界面底部的 “Resource Reservation” 约束编辑窗口右击选择 Reserve Resources，新建一条约束；
- 2. 通过输入或拖拽的方式设置位置信息；
- 3. 双击“Attribute” 栏或单击“Attribute” 栏下拉框可设置预留位置的属性，如图 3-15 所示。

注！
Name 属性用于区分不同的预留约束，不可修改该名称。

图 3-15 预留约束

	Name	Locations	Attribute
1	reserve_0	drag or type t...	ALL
			ALL
			LUT
			REG

Clock Net Constraints

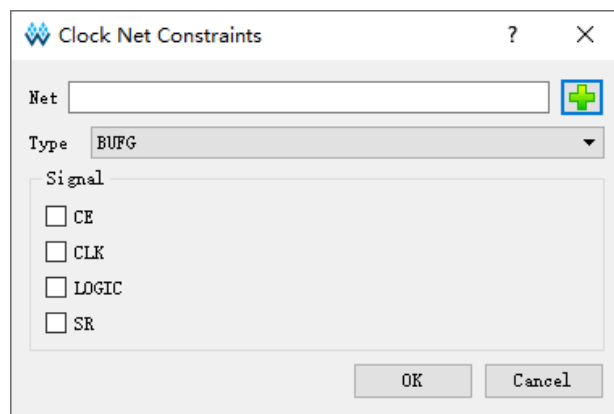
创建全局时钟分配约束，该约束的数目有限制，会对约束合法性时进行相应检查。右击选择“Clock Net Constraints”，可弹出如图 3-16 所示对话框，进行如下操作：

1. 单击“”按钮，选择对应 Net；
2. 通过“Type”下拉列表，选择“BUFG”、“BUFG[0]~[15]”、“LOCAL_CLOCK”；
3. 通过“CE”、“CLK”等复选框配置 Signal 类型，配置完成后，单击“OK”，产生约束信息，显示在主界面底部的“Clock Net Constraints”约束编辑窗口中，在编辑窗口中，双击，重新打开约束对话框，进行编辑。

注！

当 Type 选择 LOCAL_CLOCK 时 Signal 复选框为置灰不可配置状态。

图 3-16 时钟约束

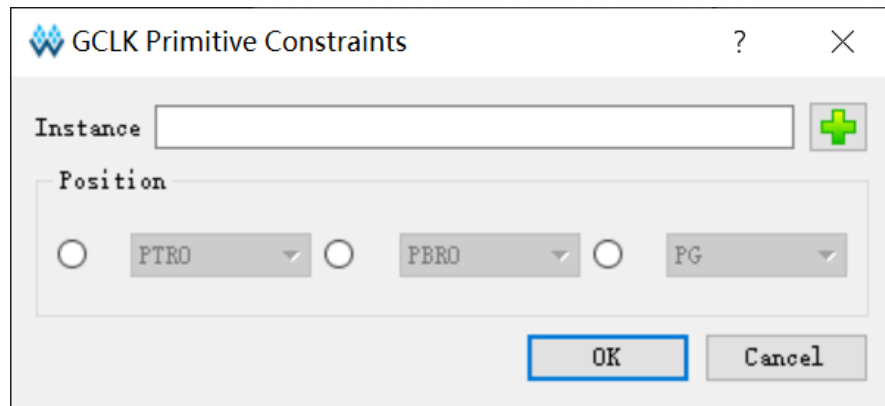


GCLK Primitive Constraints

用于创建针对全局时钟原语创建全局时钟约束。根据器件的全局时钟分布约束指定的 Instance 到具体的全局时钟，以 GW5AT-138 器件为例，在主界面底部的“GCLK Primitive Constraints”约束编辑窗口右击选择“Select GCLK Primitive”，可弹出如图 3-17 所示对话框。相关操作如下所示：

1. 通过单击“”按钮，选择相应的 GCLK 原语，若设计无 GCLK 原语则无法添加；
2. 通过 Position 下的单选框以及对应的下拉列表配置全局时钟位置；
3. 单击“OK”，产生约束信息，显示在主界面底部的“GCLK Primitive Constraints”约束编辑窗口中，在编辑窗口中，双击，重新打开约束对话框，可进行约束编辑修改。

图 3-17 创建全局时钟约束



注！

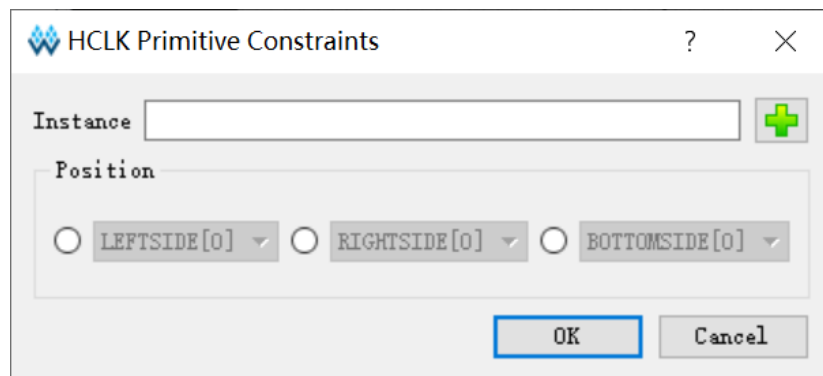
- 当选择“Instance”后，“Position”变为高亮。
- Position 根据器件不同可用 Position 不同，不同全局时钟原语可用 Position 也会不同。

HCLK Primitive Constraints

创建针对 HCLK 相关原语进行的约束，指定其约束在器件的高速时钟位置上。以 GW5AT-138 器件为例，在主界面底部的“HCLK Primitive Constraints”约束编辑窗口右击选择 **Select HCLK Primitive**，可弹出如图 3-18 所示对话框。相关操作如下所示：

1. 用户可通过单击“”按钮选择相应的 HCLK 原语，若设计中无符合的原语则无法添加；
2. 通过 Position 下的单选框以及对应的下拉列表配置高速时钟位置；
3. 单击“OK”，产生约束信息，显示在主界面底部的“HCLK Primitive Constraints”约束编辑窗口中，在编辑窗口中，双击，重新打开约束对话框，可进行约束编辑修改。

图 3-18 创建高速时钟约束



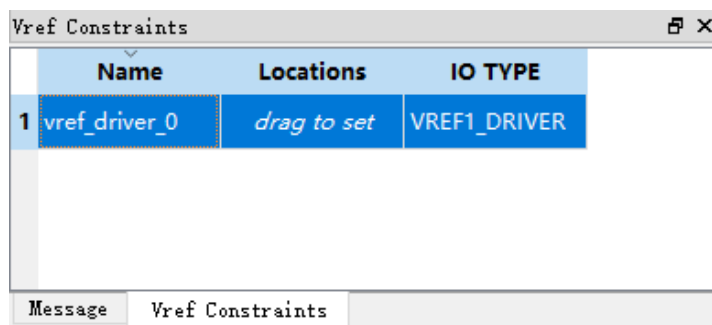
注！

- 当选择 “Instance” 后，“Position” 变为高亮。
- Position 根据器件不同可用 Position 不同，不同高速时钟原语可用 Position 也会不同。

Vref Constraints

创建新的 Vref Driver，用于配置 IO Port 的参考电压，在主界面底部的 “Vref Constraints” 约束编辑窗口中右击选择 Define Vref Driver，新建一条约束，如图 3-19 所示。

图 3-19 参考电压约束



注！

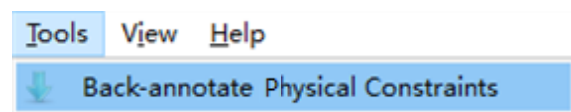
- 可通过拖拽方式指定 Vref 约束位置；
- 可通过双击修改 Vref 的名称。

Tools 菜单

Tools 菜单如图 3-20 所示。

Back-annotate Physical Constraints: 将各 Primitive 和 IO Port 布局信息反标至物理约束文件中。

图 3-20 Tools 菜单



1. 单击 “Tools > Back-annotate Physical Constraints” 弹出对象选择对话框如图 3-21 所示。Back-Annotate Physical Constraints 功能只有工程中运行 Place & Route 成功后，通过工程启动 FloorPlanner 才有效；
2. Back-annotate Physical Constraints 对话框中可以选择一个或者多个对象，单击 “OK” 按钮弹出 “Save as” 对话框，打印其布局信息至物理约束文件中；
3. 如图 3-22 所示，在 Back-annotate Physical Constraints 对话框中选择 Port 和 Port Attribute 后生成的物理约束文件。

图 3-21 Back-annotate Physical Constraints 对话框

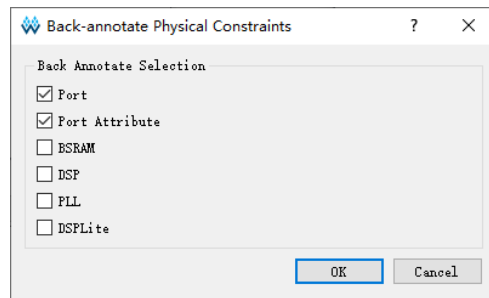


图 3-22 反标 Port 布局信息

```

1
2 IO_LOC "in1" N13;
3 IO_PORT "in1" IO_TYPE=LVC MOS33 PULL_MODE=NONE BANK_VCCIO=3.3;
4 IO_LOC "in2" G18;
5 IO_PORT "in2" IO_TYPE=LVC MOS33 PULL_MODE=NONE BANK_VCCIO=3.3;
6 IO_LOC "in3" H14;
7 IO_PORT "in3" IO_TYPE=LVC MOS33 PULL_MODE=NONE BANK_VCCIO=3.3;
8 IO_LOC "in4" J16;
9 IO_PORT "in4" IO_TYPE=LVC MOS33 PULL_MODE=NONE BANK_VCCIO=3.3;
10 IO_LOC "in5" J15;
11 IO_PORT "in5" IO_TYPE=LVC MOS33 PULL_MODE=NONE BANK_VCCIO=3.3;

```

View 菜单

View 菜单如图 3-23 所示，主要用于控制工具条、窗口的显示以及 Chip Array 和 Package View 两个视图的放大、缩小等。各子菜单介绍如下：

- **Reset Layout:** 恢复界面布局的初始设置；
- **Toolbars:** 用于控制工具栏快捷按钮的显示；
- **Windows:** 用于控制各个窗口的显示，如图 3-24 所示；
- **Zoom In:** 用于放大 Chip Array 视图或 Package View 视图；
- **Zoom Out:** 用于缩小 Chip Array 视图或 Package View 视图；
- **Zoom Fit:** 按照窗口大小缩放 Chip Array 视图或 Package View 视图。

图 3-23 View 菜单

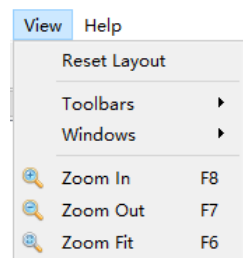
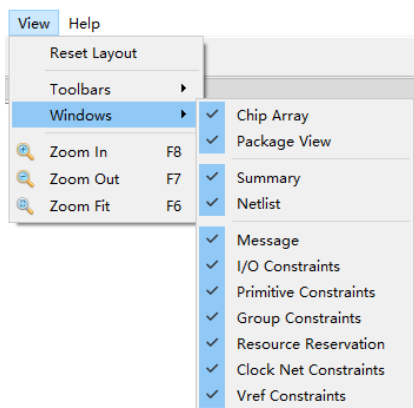


图 3-24 Windows 菜单



Help 菜单

Help 菜单用于提示软件的版本号及版权信息。

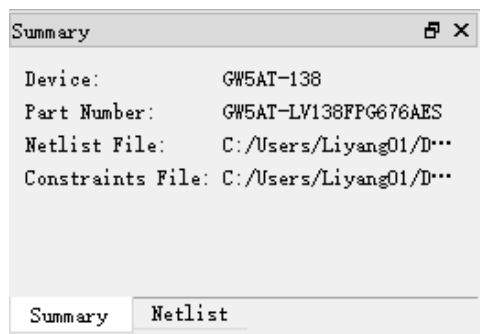
3.3.2 Summary 和 Netlist 窗口

Summary 和 Netlist 两个窗口可显示当前工程的器件信息、芯片型号信息、用户设计及约束文件的路径信息、Netlist 信息等。

Summary 窗口

Summary 窗口如图 3-25 所示，用于显示当前工程中所用的器件信息，包括 Device 和 Part Number，以及用户输入的设计文件和约束文件。

图 3-25 Summary 窗口



Netlist 窗口

Netlist 窗口如图 3-26 所示，以树形结构显示用户设计中的 Ports、Primitives、Nets 和 Module 以及对应的数量信息。

注！

- Port、Primitive 等名称采用全路径方式进行显示，默认按字母升序排序；
- Port 和 Net 的显示采用 Bus 和非 Bus 相结合的显示方式，如图 3-27 所示；

- Module 采用层级的方式显示, 各 Module 后可显示各 Module 中各类型的 Instance 数目, 如图 3-28 所示;

图 3-26 Netlist 窗口

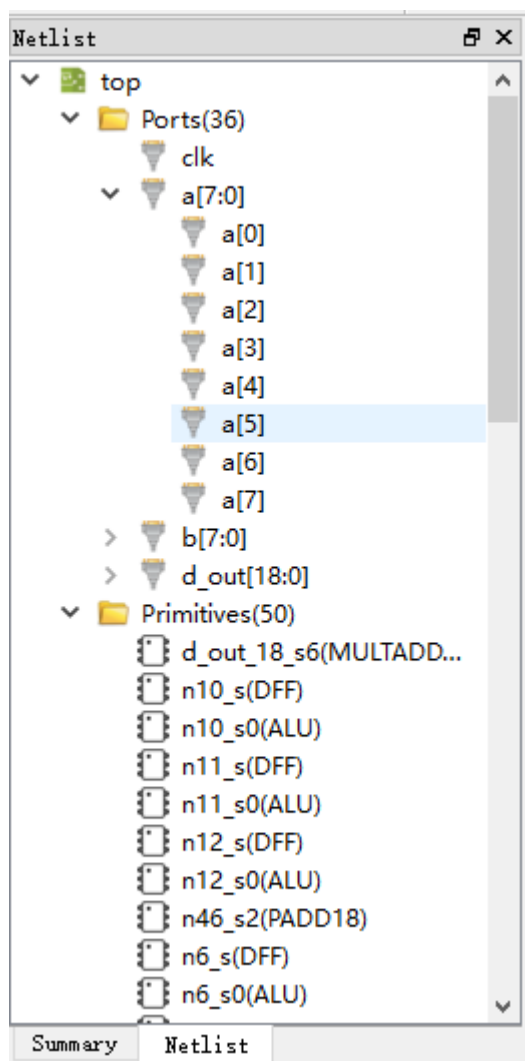


图 3-27 BUS 和非 BUS 结合显示

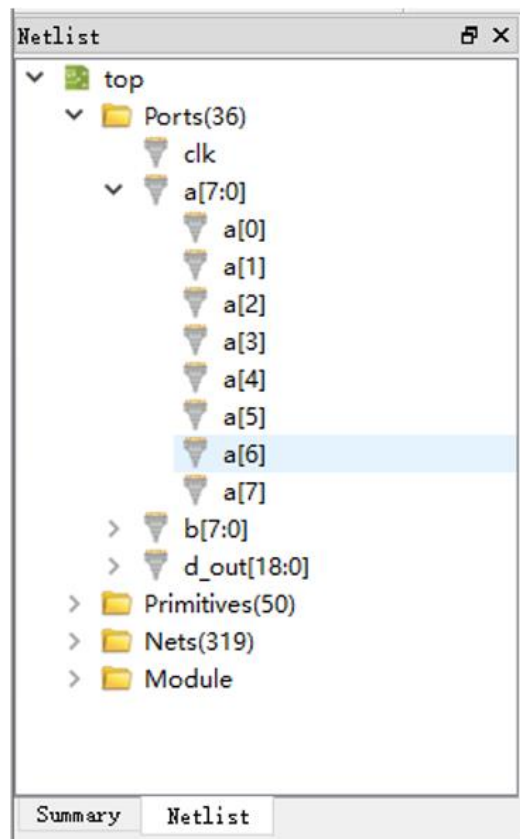
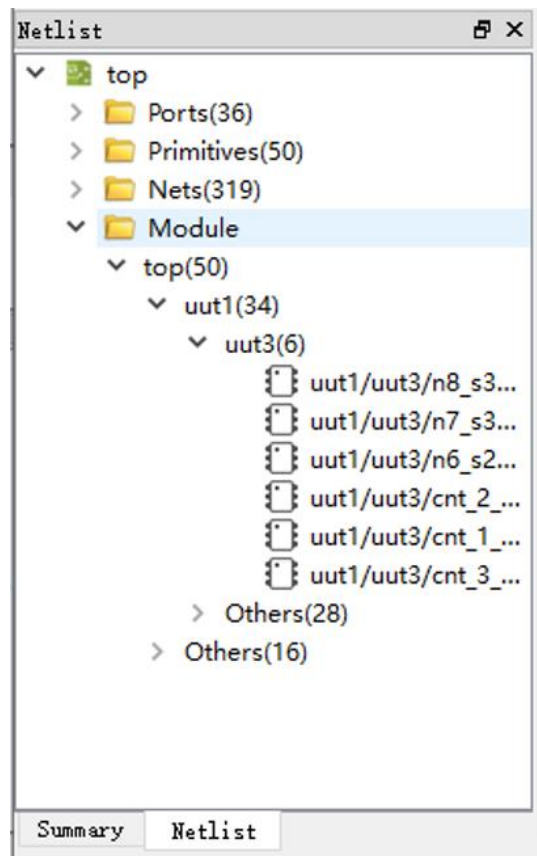


图 3-28 层级显示



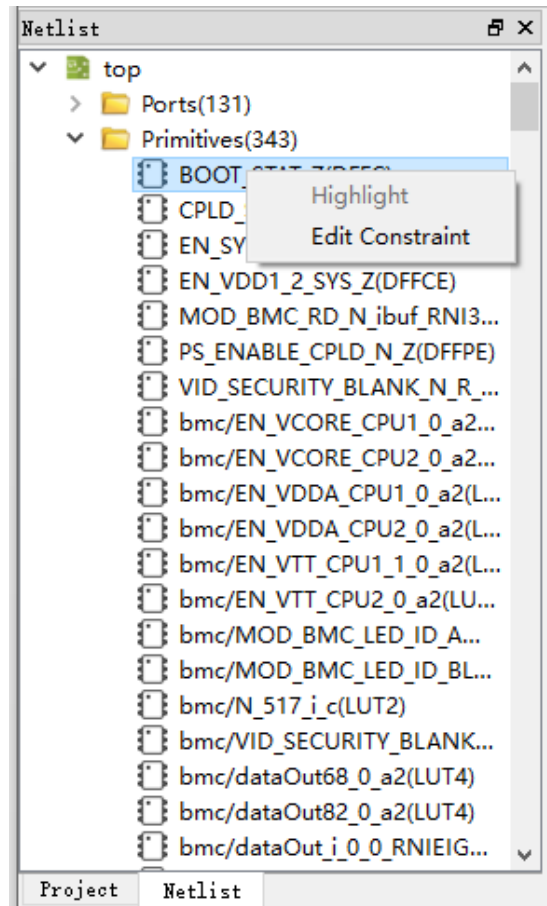
Netlist 窗口提供右键菜单功能，具有如下功能：

- **Highlight:** 可实现在 Chip Array 中高亮显示对应的约束位置；
- **Edit Constraint:** 编辑对应约束信息的功能。

注！

如当前 Primitive 或 Port 无位置约束，则高亮显示功能不可用，如图 3-29 所示。

图 3-29 Netlist 右键功能



3.3.3 Package View 窗口

以 GW5AT-138-FCPBGA676A 为例，Package View 窗口如图 3-30 所示，该窗口以器件 package 信息为基础显示器件的封装信息，显示用户 I/O、电源、地等管脚。将鼠标放置某个位置上时，会悬浮显示该位置的 I/O 信息，包括 I/O 的 Type、Bank 以及 LVDS 信息等。

图 3-30 GW5AT-138-FPBGA676A Package view 窗口



用户 I/O、电源、地管脚使用不同的符号和颜色来区分。不同 BNAK 的 IO 引脚的颜色不同，图中管脚符号定义如下所示：

- “” 表示用户 I/O；
- “” 表示 VCCIO；
- “” 表示 VSS。

Package View 支持右键菜单如图 3-31 所示，相关功能如下：

- Zoom In: 放大 Package View 视图；
- Zoom Out: 缩小 Package View 视图；
- Zoom Fit: 按照窗口大小缩放 Package View 视图；
- Show Differential IO Pairs: 显示差分对，如图 3-32 所示，红线相连的为一对差分对；
- Top View: Package View 以顶部视图进行显示，默认以顶部视图进行显示。如图 3-33 所示为 GW5AT-138-FCPBGA676A 封装的顶部视图，坐标原点在左上角；

- Bottom View: Package View 以底部视图进行显示。如图 3-34 所示为 GW5AT-138-FCPBGA676 封装的底部视图，坐标原点在右上角。

图 3-31 Package View 右键功能

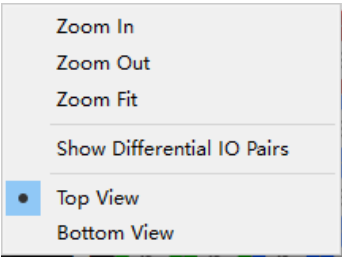


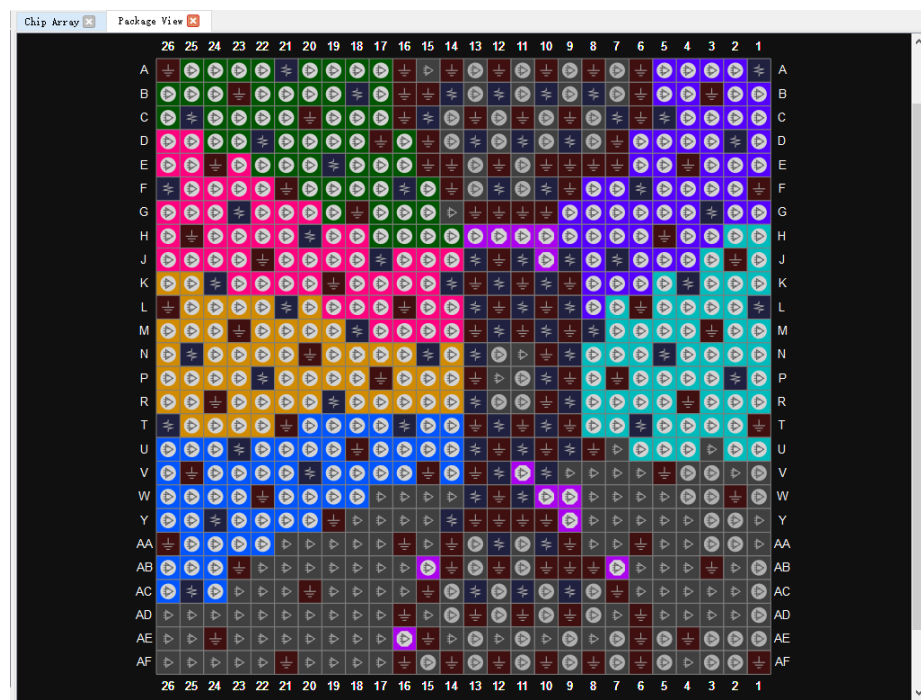
图 3-32 差分对显示



图 3-33 Top View



图 3-34 Bottom View



Package View 支持 IO Port 约束位置的显示, 可以通过从 Netlist 窗口或底部 I/O Constraints 窗口中将 IO Port 拖拽到 Package View 窗口来约束 IO Port 的位置。拖拽时鼠标处会显示拖拽 Port 的名称, 并且不可约束管脚呈现置灰不可拖拽状态。

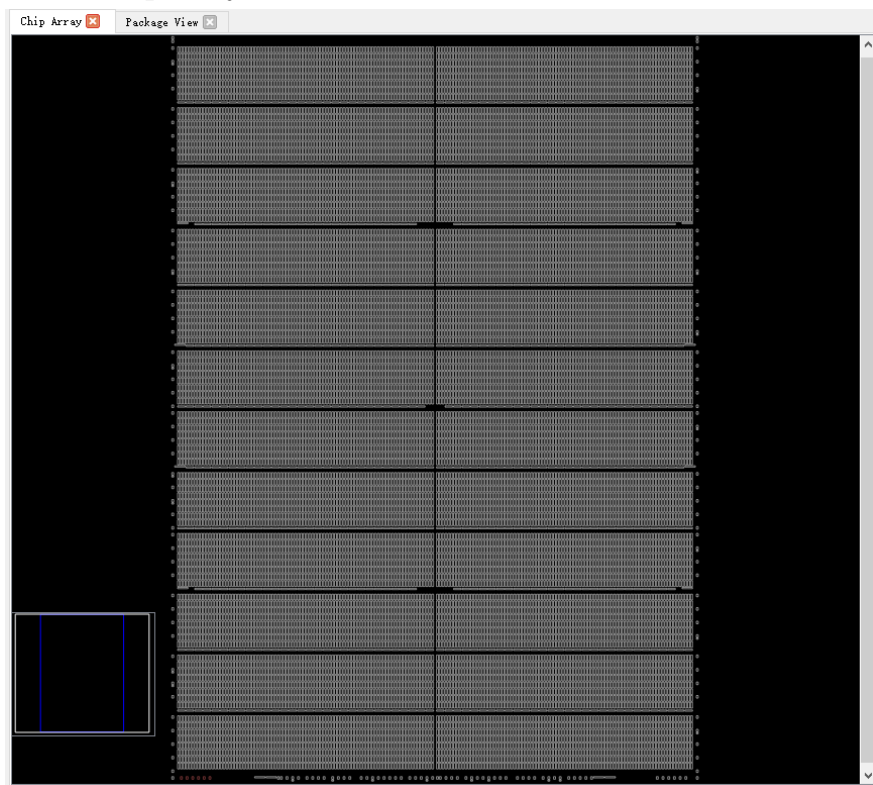
3.3.4 Chip Array 窗口

FloorPlanner 的 Chip Array 窗口如图 3-35 所示, Chip Array 窗口根据芯片的行列信息显示芯片的 I/O、CFU、DSP、PLL、BSRAM、DQS 等资源的分布, 实现对所有约束位置的实时显示, 支持放大、缩小、复制位置、悬停显示、拖拽等功能。

其中, I/O 是器件裸片的所有 I/O 位置, 且会以不同颜色区分 I/O:

- 白色: 该封装中封装出的 I/O 位置;
- 红色: 该封装中未封装的 I/O 位置。

图 3-35 Chip Array 窗口



Chip Array 分为网格模式、宏单元模式、原语模式三种显示模式。

- 网格模式: 以 GRID 为单位宏观显示约束位置, 如图 3-36 所示;
- 宏单元模式: 以 CLS、IOBLK 等单位显示约束位置, 如图 3-37 所示;
- 原语模式: 以 REG、LUT 等单位显示约束位置, 如图 3-38 所示。

图 3-36 网格模式约束

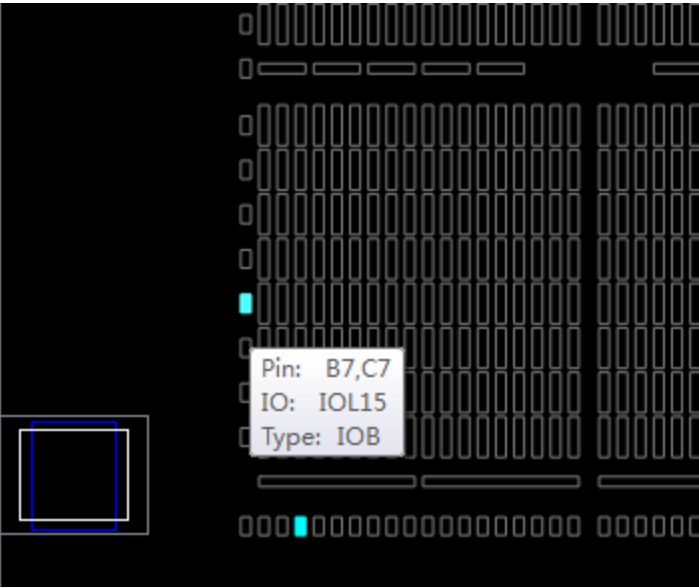


图 3-37 宏单元模式约束

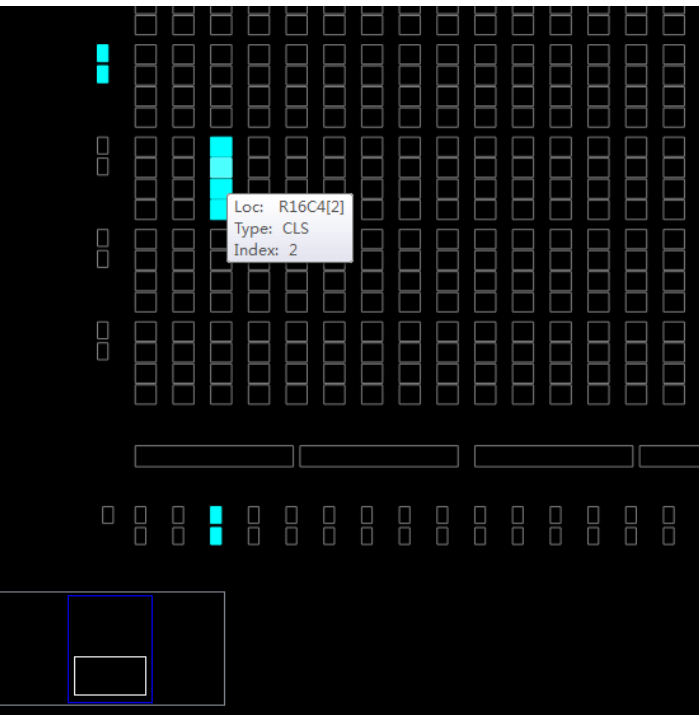
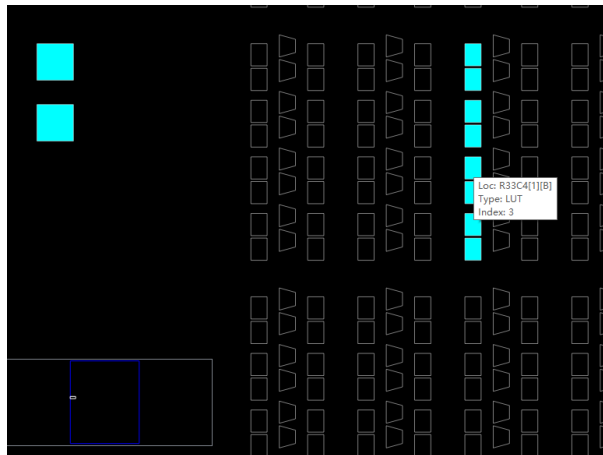


图 3-38 原语模式约束



Chip Array 支持以下拖拽功能：

- 从 Netlist 窗口拖拽到 Array 窗口，用于产生约束、指定约束位置；
- 从约束编辑窗口拖拽到 Array 窗口，用于指定约束位置。

Chip Array 窗口内置 chip 子窗口，用于实时显示当前视窗相对于整个器件的位置，拖动 chip 子窗口中的白色框，Chip Array 的视窗将跟随移动。同时，Chip Array 窗口采用不同颜色区分约束类型、显示约束位置，各颜色的含义分别介绍如下：

- 白色：用于显示处于选择状态或正在高亮显示的约束位置；
- 深蓝色：用于显示预留约束的位置信息，表示该位置不能再被占用；
- 浅蓝色：用于显示 I/O 和 Primitive 约束在某个 GRID 或 range 内，界面上显示为浅蓝色。

Chip Array 窗口支持右键菜单，具有如下功能：

- Zoom In: 放大 Chip Array 视图；
- Zoom Out: 缩小 Chip Array 视图；
- Zoom Fit: 按照窗口大小缩放 Chip Array 视图；
- Show Constraints View: 显示 Chip Array 的 instance 约束视图；
- Show Place View: 显示 Chip Array 的 instance 布局视图，只有在工程运行完 Place & Route 后启动 FloorPlanner 才有效，否则置灰；
- Show Multi-View: 同时显示 Chip Array 的 instance 约束和布局的复合视图，只有在工程运行完 Place & Route 后启动 FloorPlanner 才有效，否则置灰；
- Show In-Out Connection: 在 Place View 中显示和选中 instance 输入输出连接的 instance 位置，只有在 Chip Array 窗口是 Show Place View >

All Instance 视图中选中某个 instance 才能够使用，否则置灰；

- **Show In Connection:** 在 Place View 中显示和选中 instance 输入连接的 instance 位置，只有在 Chip Array 窗口是 Show Place View > All Instance 视图中选中某个 instance 才能够使用，否则置灰；
- **Show Out Connection:** 在 Place View 中显示和选中 instance 输出连接的 instance 位置，只有在 Chip Array 窗口是 Show Place View > All Instance 视图中选中某个 instance 才能够使用，否则置灰；
- **Unhighlight All:** 对于选中位置或者区域消除高亮；
- **Copy Location:** 复制选中的位置或者区域，若 Chip Array 窗口中有 GRID、Block 等处于选中状态，则右键菜单中的“Copy Location”功能可用，否则功能不可用，如图 3-39 所示。

在 Show Place View 中，可对 Lut、Reg 的密度进行显示，如图 3-40 所示，详情如下：

- **ALL Instance:** 显示所有 Instance 的 place 情况，5 个以内呈淡绿色、6-10 个呈绿色、10 个以上呈深绿色；
- **All Lut:** 只显示所有 Lut 的 place 情况，2 个以内呈淡绿色、3-4 个呈绿色、4 个以上呈深绿色；
- **All Dff:** 只显示所有 Reg 的 place 情况，2 个以内呈淡绿色、3-4 个呈绿色、4 个以上呈深绿色。

在 Show Place View > ALL Instance 可以查看设计中所有 instance 的布局情况：

- 在 Chip Array 窗口中鼠标悬浮至 instance 布局位置，可显示该位置具体布局的 instance 名称，如图 3-41 所示；
- 在 Netlist 窗口中选中某个具体的 instance 右键选择 Highlight，该 instance 的布局位置会在 Chip Array 窗口中高亮显示，如图 3-42 所示。

注！

通过“Ctrl”键+鼠标左键拖动，可选取区域，右击选择 Copy Location，可复制所选区域的位置信息，复制的位置可直接粘贴到任意约束编辑窗口中。

图 3-39 Chip Array 右键功能

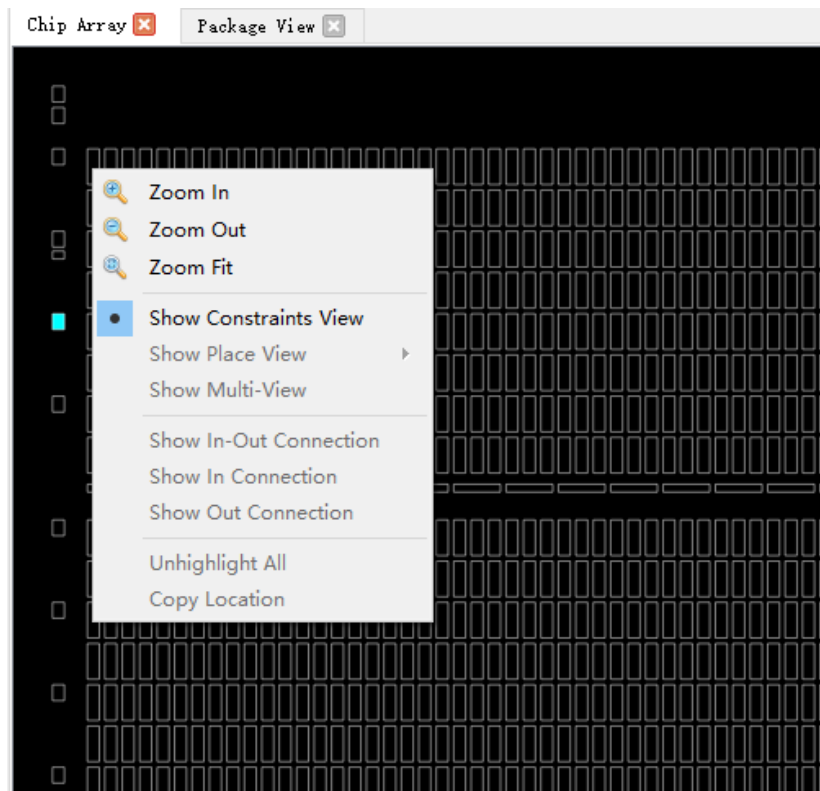


图 3-40 Show Place View 显示

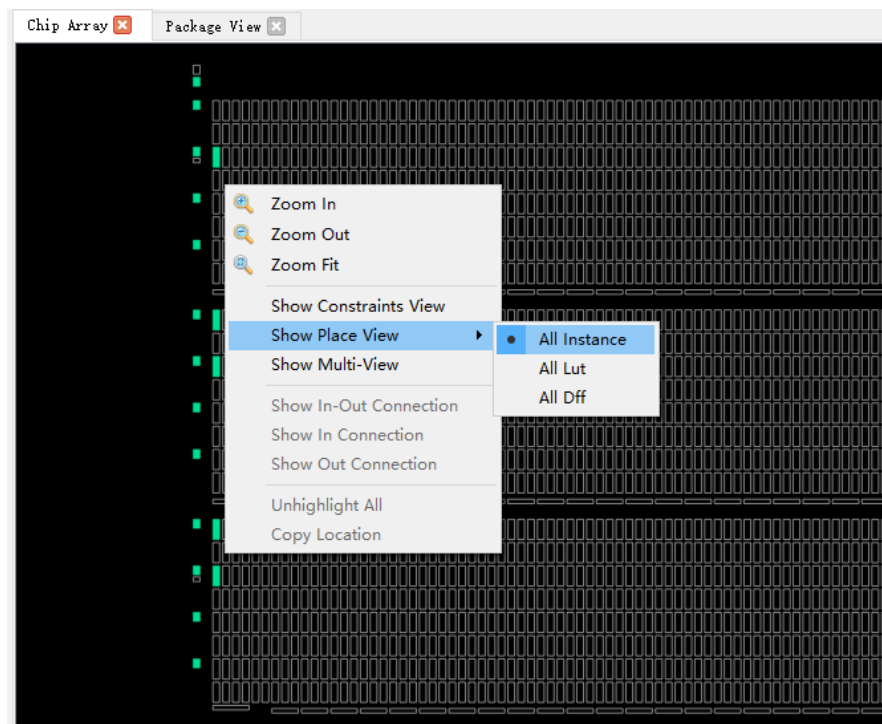


图 3-41 鼠标悬浮显示

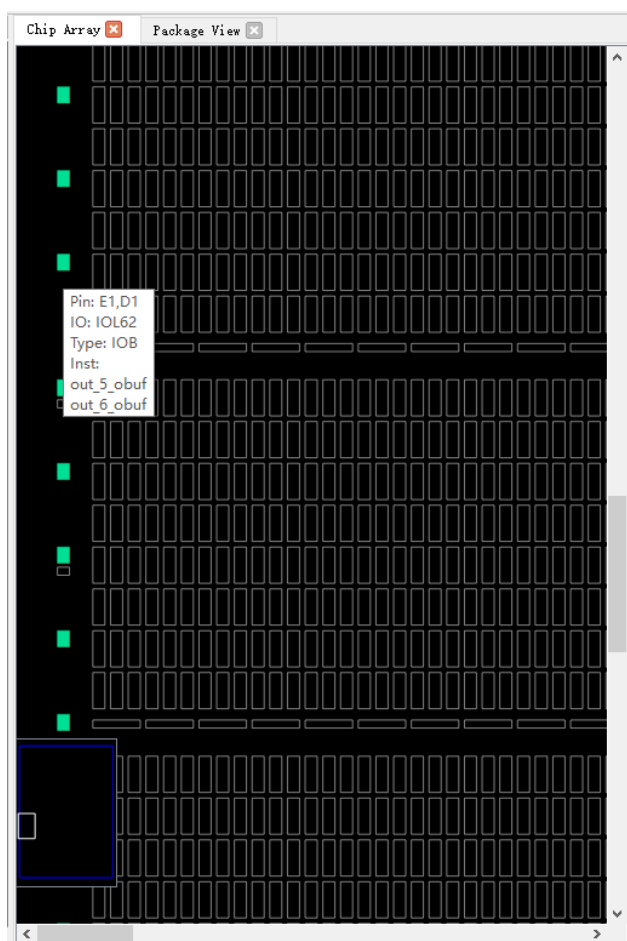
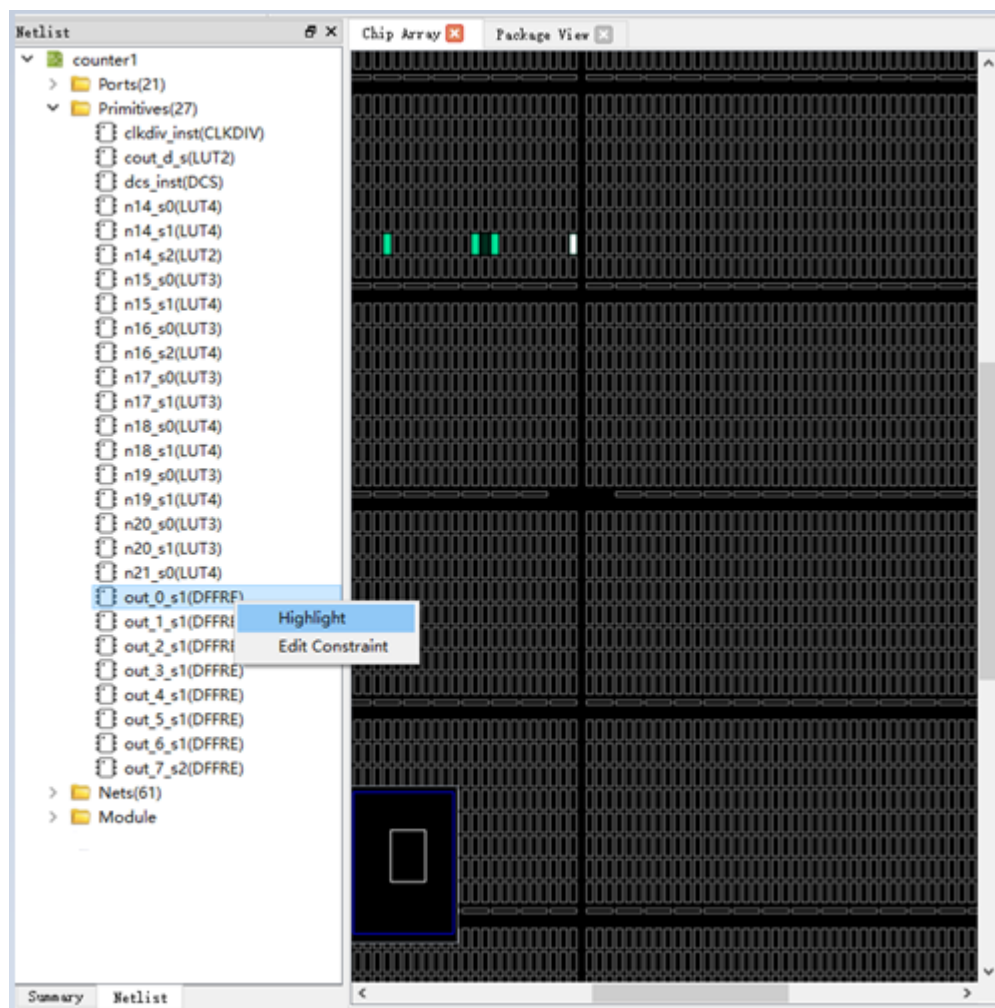


图 3-42 右键高亮



3.3.5 Constraint 编辑窗口

Constraint 编辑窗口包含 “I/O Constraints”、“Primitive Constraints”、“Group Constraints” 等 8 个编辑窗口，用于显示各约束的详细信息，并提供约束编辑功能和位置拖拽功能，各窗口分别介绍如下：

I/O Constraints

I/O Constraints 是对设计的 port 进行管脚约束。I/O 约束窗口如图 3-43 所示，各功能如下：

- 显示用户设计中所有 IO Port 的属性及约束信息，如 Port 的 Direction、Bank、IO Type、Pull Mode 等；
- 提供约束位置、属性等编辑功能；
- 可通过拖拽、双击等方式改变约束信息。

注！

- I/O 的位置可以通过拖拽的方式进行设置，也可以双击输入；
- 在拖拽 IO 过程中会显示所拖拽的 IO 名称；
- 将 IO 拖拽至 Chip Array 窗口中时，可放置的位置变亮，不可放置的位置颜色亮度不变；
- 将 IO 拖拽至 Package View 窗口中时，可放置位置亮度不变，不可放置位置变暗；
- 设置完成后，在 Chip Array 窗口中约束的位置变为浅蓝色高亮，在 Package View 窗口中约束的位置变为橙色高亮。

窗口提供右键菜单功能，详情如下：

- Unplace: 取消放置；
- Reset Properities: 复位 Port 属性设置；
- Highlight: 高亮显示约束位置；
- IO Type: 设置电平标准；
- Drive: 设置驱动电流；
- Pull Mode: 设置上拉模式；
- Pull Strength: 设置上拉强度；
- PCI Clamp: 设置 PCI 协议的开关；
- Hysteresis: 设置迟滞量；
- Open Drain: 设置开漏电路的开关；
- Vref: 设置参考电压；
- Single Resistor: 设置单端电阻的开关；
- Diff Resistor: 设置差分电阻的开关；
- Bank Vccio: 设置 BANK 电压；
- Ctle: 连续时间线性均衡器，用于补偿信道衰减；
- Slew Rate: 设置电压转换速率。

注！

- 右键菜单支持用户批量修改 Port 属性的功能，用户可选择多个 Port，若多个 Port 有相同的属性值可配置，则通过右键菜单可统一进行配置；详细属性设置标准请参考 [DS981, Arora V 138K & 75K FPGA 产品数据手册](#)。
- Slew Rate 属性支持器件：GW5A(T)-60、GW5A(N)(R)T-15。

图 3-43 I/O 约束窗口

I/O Constraints											
	Port	Direction	Diff Pair	Location	Bank	Exclusive	IO Type	Drive	Pull Mode	PCI Clamp	Hys ^
2	cin	input				False	LVC MOS18	N/A	UP	N/A	N
3	clk	input				False	LVC MOS18	N/A	UP	N/A	N
4	clk0	output				False	LVC MOS18	8	UP	N/A	I
5	cout	output				False	LVC MOS18	8	UP	N/A	I
6	data[0]	input				False	LVC MOS18	N/A	UP	N/A	N
7	data[1]	input				False	LVC MOS18	N/A	UP	N/A	N

Primitive Constraints

Primitive Constraints 是约束设计中原语的位置,原语约束窗口如图 3-44 所示, 功能如下:

- 用于显示当前所有 Primitive 约束的名称、类型、位置以及 Exclusive 信息;
- 提供编辑功能, 该窗口提供右键菜单功能, 用于提供高亮显示约束位置、删除和添加约束的功能。

注!

- 可通过拖拽的方式或双击输入的方式修改位置信息;
- 可通过双击设置 Exclusive 属性;
- 在 Primitive 约束位置进行手动输入时, 会对位置进行语法检查及合法性检查, 错误提示对话框如图 3-11 和图 3-12 所示。

图 3-44 原语约束窗口

Primitive Constraints			
	Primitive	Type	Locations Exclusive
1	ALU_check	ALU	P4C10 False

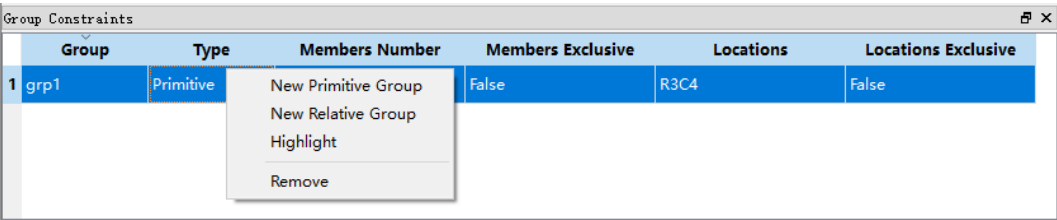
Group Constraints

Group Constraints 是对设计中的 I/O 和部分原语进行组约束, 组约束窗口如图 3-45 所示, 功能如下:

- 用于显示当前所有组约束的名称、类型、包含的 Primitive 个数、位置以及 Exclusive 信息, 包含 Primitive 和 Relative 两种 Group 的显示; 如图 3-10 和图 3-14 所示, 双击对应的 Group 条目, 打开对话框, 可实现约束信息的编辑修改功能;
- 该窗口提供右键菜单功能, 用于提供高亮显示约束位置、删除和添加约

束的功能。

图 3-45 组约束窗口



Resource Reservation

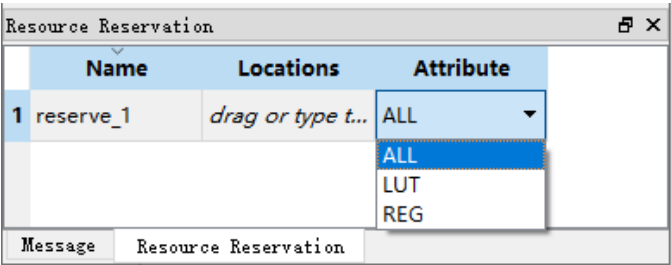
Resource Reservation 是对当前封装中的可用资源进行预留约束，预留约束窗口如图 3-46 所示，功能如下：

- 用于显示当前所有预留约束的位置信息；
- 该窗口提供右键菜单功能，用于提供高亮显示约束位置、删除、添加约束的功能；
- Name 属性用于区分各资源预留约束，用户不能进行修改。

注！

可通过拖拽或双击输入修改位置信息。

图 3-46 预留约束窗口



Clock Net Constraints

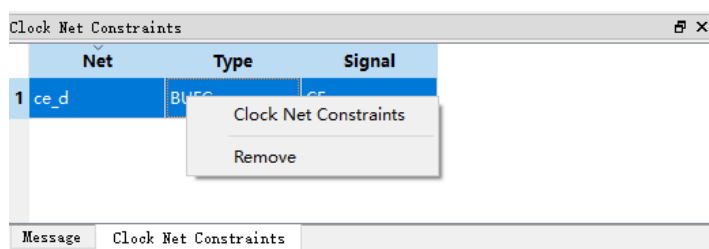
Clock Net Constraints 是对设计中的 net 进行全局时钟分配约束，时钟分配约束窗口如图 3-47 所示，功能如下：

- 用于显示当前所有时钟分配约束的相关信息；
- 该窗口提供右键菜单功能，用于提供添加、删除时钟分配约束的功能。

注！

- 双击约束可进行编辑修改；
- CLOCK Net 约束无位置信息，不支持拖拽功能；
- 新建全局时钟约束的窗口如图 3-16 所示。

图 3-47 时钟约束窗口



GCLK Primitive Constraints

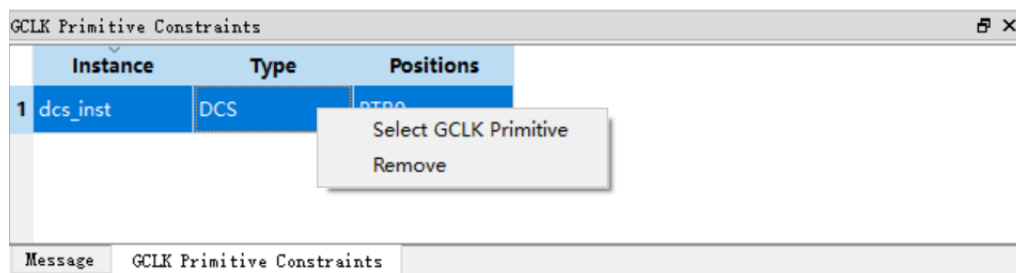
GCLK Primitive Constraints 是对全局时钟原语进行约束，全局时钟约束窗口如图 3-48 所示，功能如下：

- 用于显示所有的全局时钟约束，包括 Instance 名称、类型以及象限位置；
- 窗口支持右键菜单功能，用于添加新的全局时钟约束和删除已有约束。

注！

新建全局时钟约束的窗口如图 3-17 所示。

图 3-48 全局时钟约束窗口



HCLK Primitive Constraints

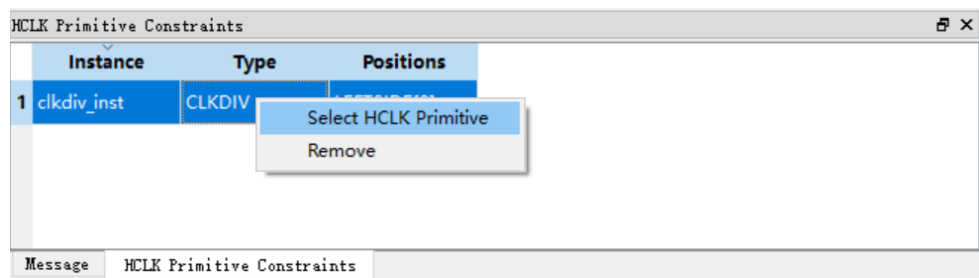
HCLK Primitive Constraints 是对设计中的高速时钟原语进行高速时钟约束，高速时钟约束窗口如图 3-49 所示，功能如下：

- 用于显示针对高速时钟相关的 Instance 的位置约束，包括 Instance 名称、类型以及高速时钟位置；
- 窗口支持右键菜单功能，用于添加新的高速时钟约束和删除已有约束。

注！

新建全局时钟约束的窗口如图 3-18 所示。

图 3-49 高速时钟约束窗口



Vref Constraints

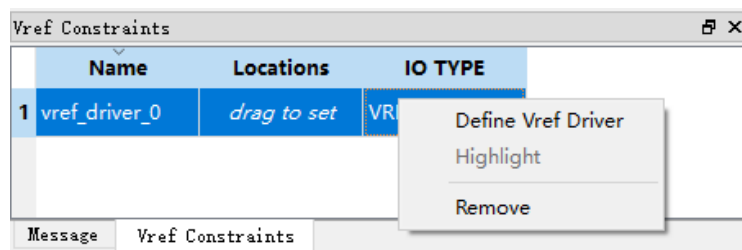
Vref Constrains 是约束所在 bank 的外部参考电压，Vref 约束窗口如图 3-50 所示，功能如下：

- 用于显示用户自定义的 Vref Driver 信息，用户可自定义 Vref 的名称、位置信息；
- 窗口支持右键菜单功能，用于高亮显示约束位置、添加、删除约束信息。

注！

位置信息只能通过拖拽的方式进行设置。

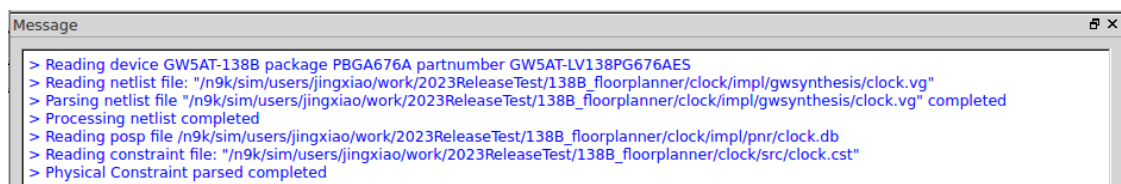
图 3-50 Vref 约束窗口



3.3.6 Message 窗口

Message 窗口如图 3-51 所示，窗口提供输出结果的显示。

图 3-51 Message 窗口



4 FloorPlanner 使用

FloorPlanner 可以新建和编辑约束，生成供布局布线流程使用的物理约束文件。

4.1 新建约束文件

FloorPlanner 可输出新建的物理约束文件，亦可输出修改后的物理约束文件，操作步骤如下所示：

1. 根据 3.2 启动所述，启动 FloorPlanner；
2. 单击“File > New”，打开“New”对话框，如图 4-1 所示；

注！

亦可通过以下两种方式打开“New”对话框：

- 使用快捷键 **Ctrl + N**；
- 单击工具栏上的“New”图标。

3. 输入工程的网表文件，选择器件类型，单击“OK”。

图 4-1 新建约束文件

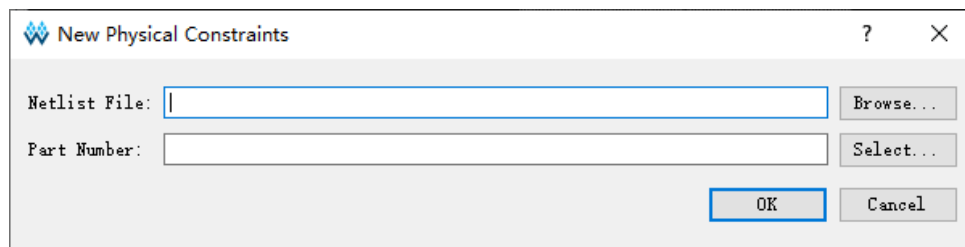
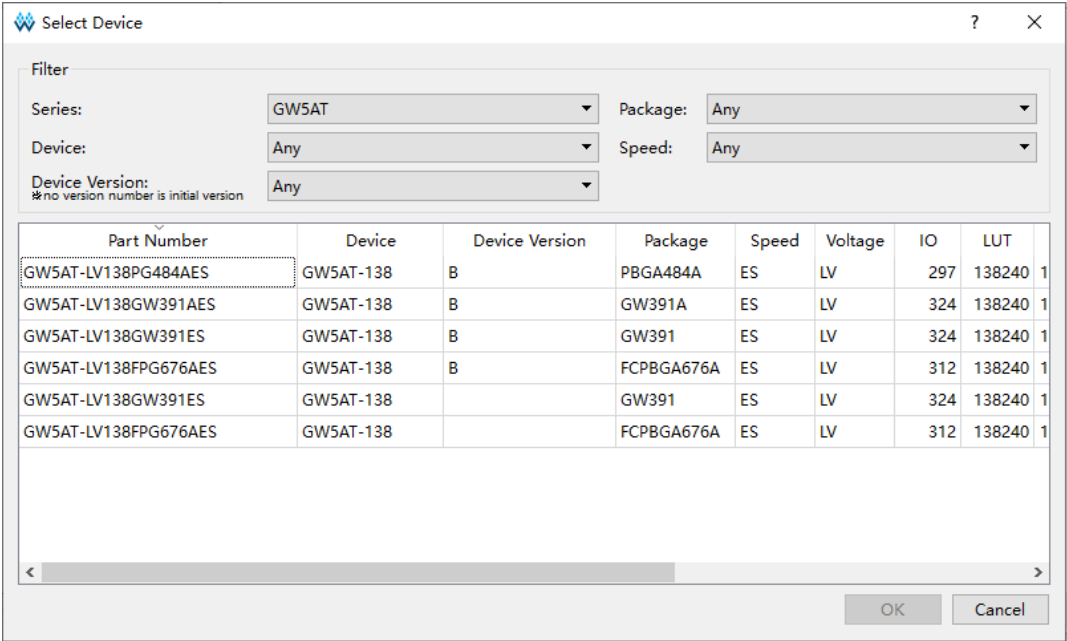


图 4-2 选择器件



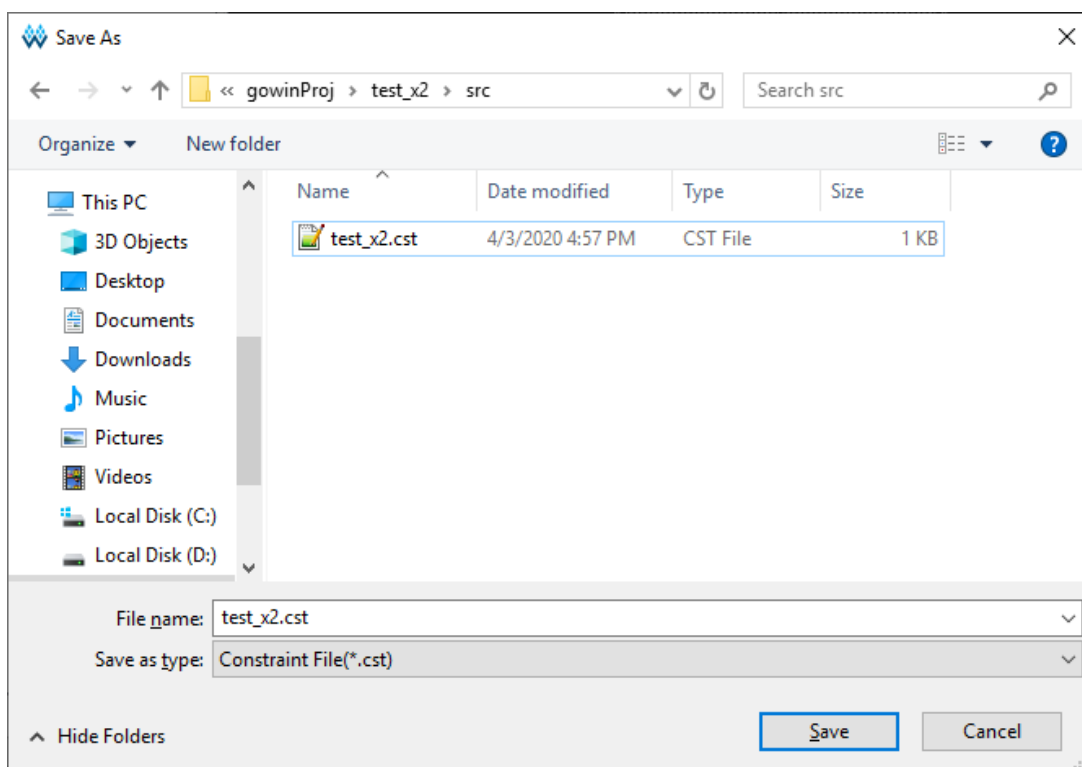
注！

- Select ...按钮用于选取器件、封装信息，支持高云半导体所有的 FPGA 器件，如图 4-2 所示；
- 启动 FloorPlanner 采用 3.2 启动介绍中的第三种方式。

新建物理约束，在 FloorPlanner 主界面中可进行如下操作：

1. 用户通过拖拽等方式分配管脚位置；
2. 单击工具栏中的“Save”图标，即可输出约束文件；
3. 在弹出的“Save”对话框中，可修改文件名，如图 4-3 所示。

图 4-3 保存输出文件



4.2 编辑约束文件

FloorPlanner 支持 I/O 约束、原语约束、组约束、资源预留约束、全局时钟分配约束、参考电压约束等的创建。可通过 **Constraints** 菜单编辑生成约束，详情请参考 [3.3.1 菜单栏](#)。

注！

亦可通过其他方式创建约束，本节主要以拖拽的方式为例，介绍如何通过拖拽编辑生成约束。

4.2.1 编辑约束示例

以用户设计 counter.v 为例，演示如何编辑各类型约束：

```
module counter1(out, cout, data, load, cin, clk, clko);  
output [7:0] out;  
output cout;  
output clko;  
input [7:0] data;  
input load, cin, clk;
```

```
reg [7:0] out;
always @(posedge clk)
begin
    if (load)
        out = data;
    else
        out = out + cin;
end
assign cout = &out & cin;
wire clkout;
CLKDIV clkdiv_inst (
    .CLKOUT(clkout),
    .HCLKIN(clk),
    .RESETN(1'b1),
    .CALIB(1'b0)
);
defparam clkdiv_inst.DIV_MODE = "2";

DCS dcs_inst (
    .CLKOUT(clko),
    .CLKSEL(4'b0000),
    .CLKIN0(clkout),
    .CLKIN1(clkout),
    .CLKIN2(clkout),
    .CLKIN3(clkout),
    .SELFORCE(1'b0)
);
```

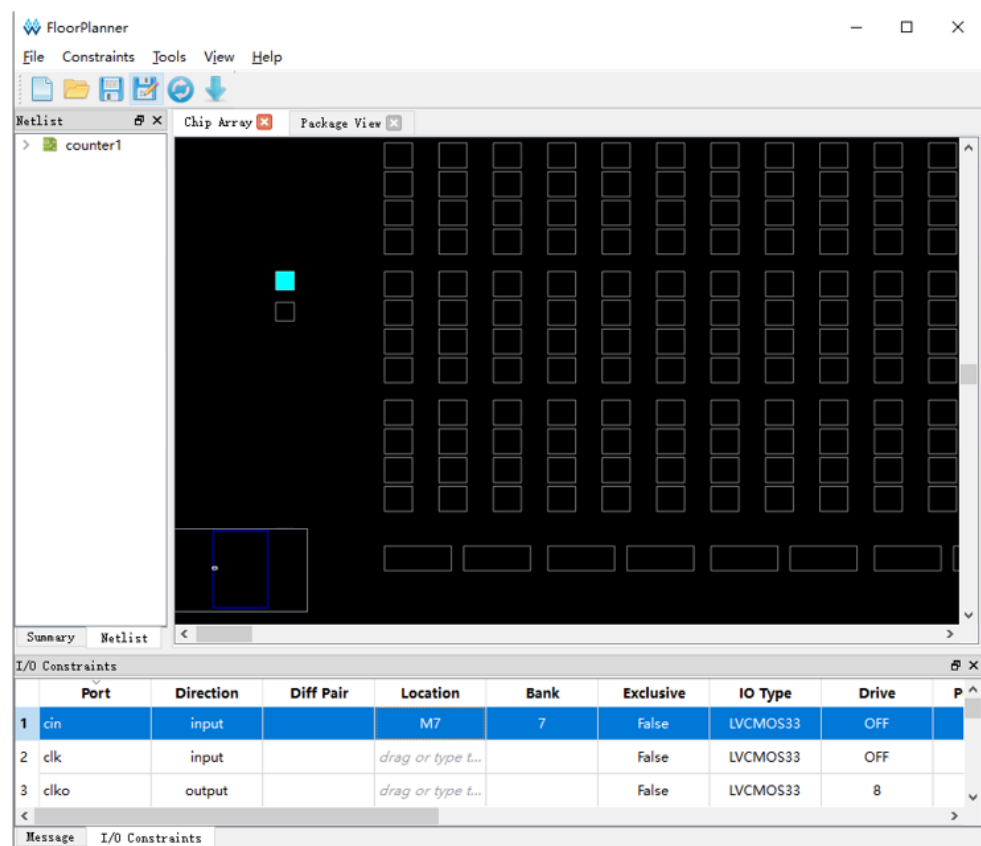
```
defparam dcs_inst.DCS_MODE = "RISING";
endmodule
```

4.2.2 编辑 I/O 约束

拖至 Chip Array 创建 I/O Constraints，步骤如下：

1. 单击 IO Constraints 编辑窗口，放大 Chip Array 视图至宏单元模式；
2. 选中 Port “cin” 拖拽至 Chip Array 窗口中的 M7 位置，如图 4-4 所示；
3. Port “cin” 的 Location 信息显示为 M7。

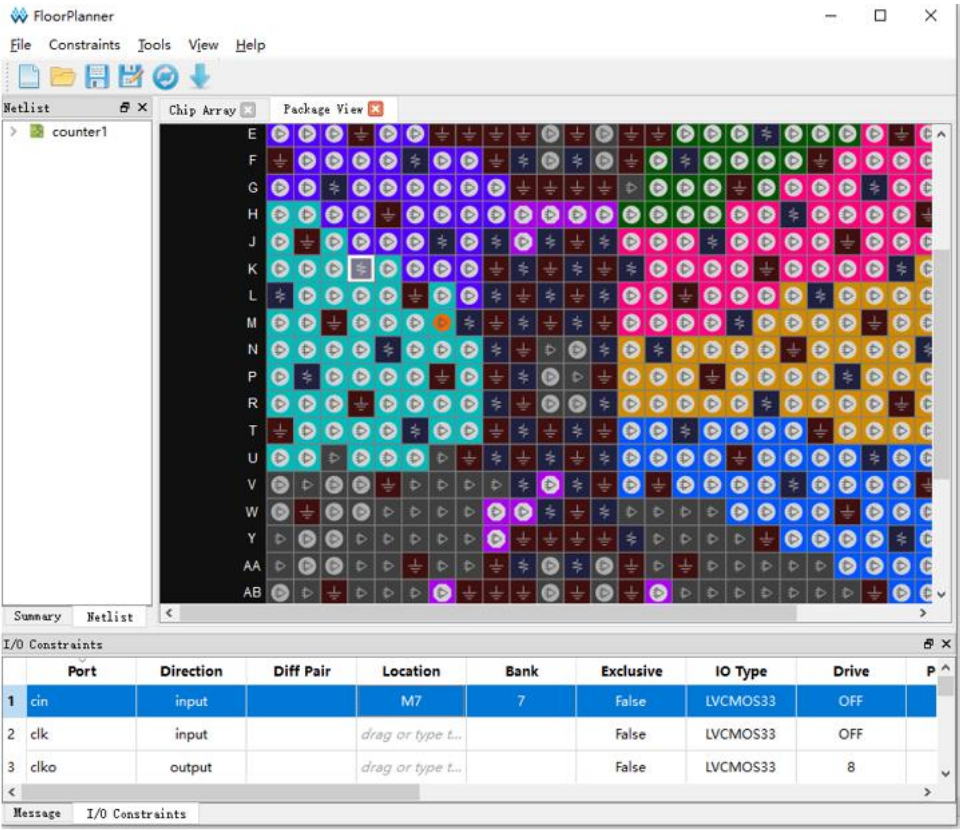
图 4-4 拖拽到 Chip Array 创建 I/O Constraints



拖至 Package View 创建 I/O Constraints，步骤如下：

1. 单击至 IO Constraints 编辑窗口；
2. 选中 Port “cin” 拖拽至 Package View 窗口中的 M7 位置，如图 4-5 所示；
3. Port “cin” 的 Location 信息显示为 M7。

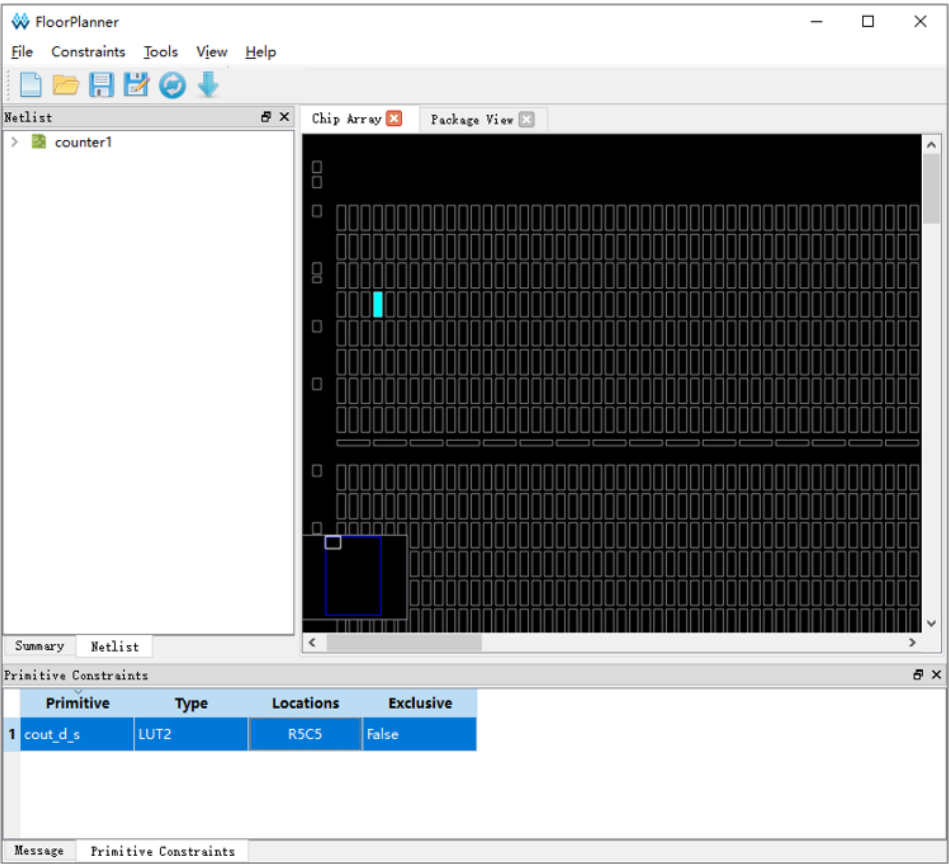
图 4-5 拖至 Package View 创建 I/O Constraints



4.2.3 编辑原语约束

- 1. 在 Primitive Constraints 编辑窗口，右击选择 “Select Primitives”，弹出 “Select Primitives” 对话框后，选择 Primitive “cout_d_s”，单击 “OK”；
- 2. 选中新创建的 Primitive 约束，拖拽至 Chip Array 窗口中的 “R5C5” 位置，如图 4-6 所示；
- 3. Primitive “cout_d_s” 的 Location 信息显示为 R5C5。

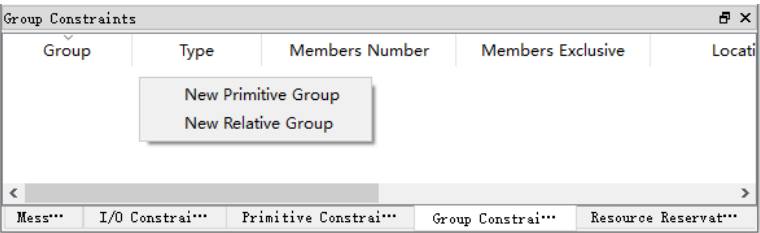
图 4-6 拖拽到 Chip Array 创建 Primitive Constraints



4.2.4 编辑组约束

如图 4-7 所示，在 Group Constraints 编辑器中，可通过右击选择菜单，创建 Primitive Group 和 Relative Group。

图 4-7 Group Constraints 编辑器右键菜单



编辑原语组约束

- 1. 在 Group Constraints 编辑窗口中，右击选择 “New Primitive Group”，弹出 “New Primitive Group” 对话框；
- 2. 输入 Group Name “grp1”，单击 “+” 弹出 “Select Primitives” 对话框；
- 3. 选取所要设置的 Primitive “n17_s0”、“cout_d_s”，单击 “OK”，加入 Members 列表；

4. 在 Locations 输入所要约束的位置 “R5C10”，如图 4-8 所示；
5. 在 New Primitive Group 对话框中单击“OK”，可创建一条 Primitive Group Constraints，如图 4-9 所示。

图 4-8 创建 Primitive Group Constraints

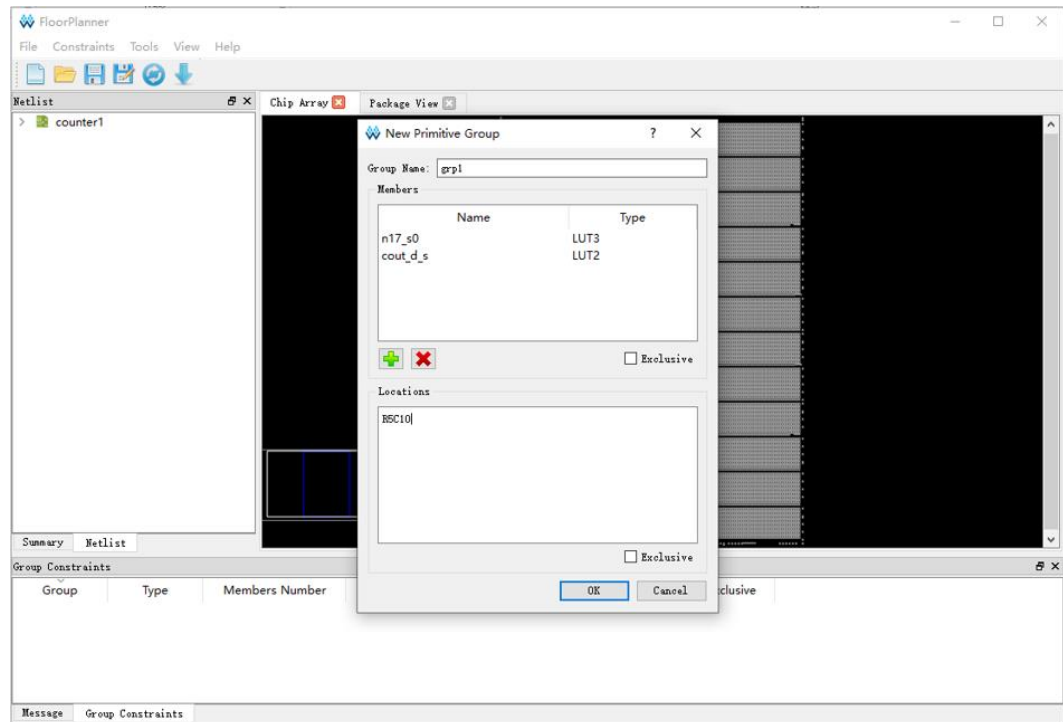
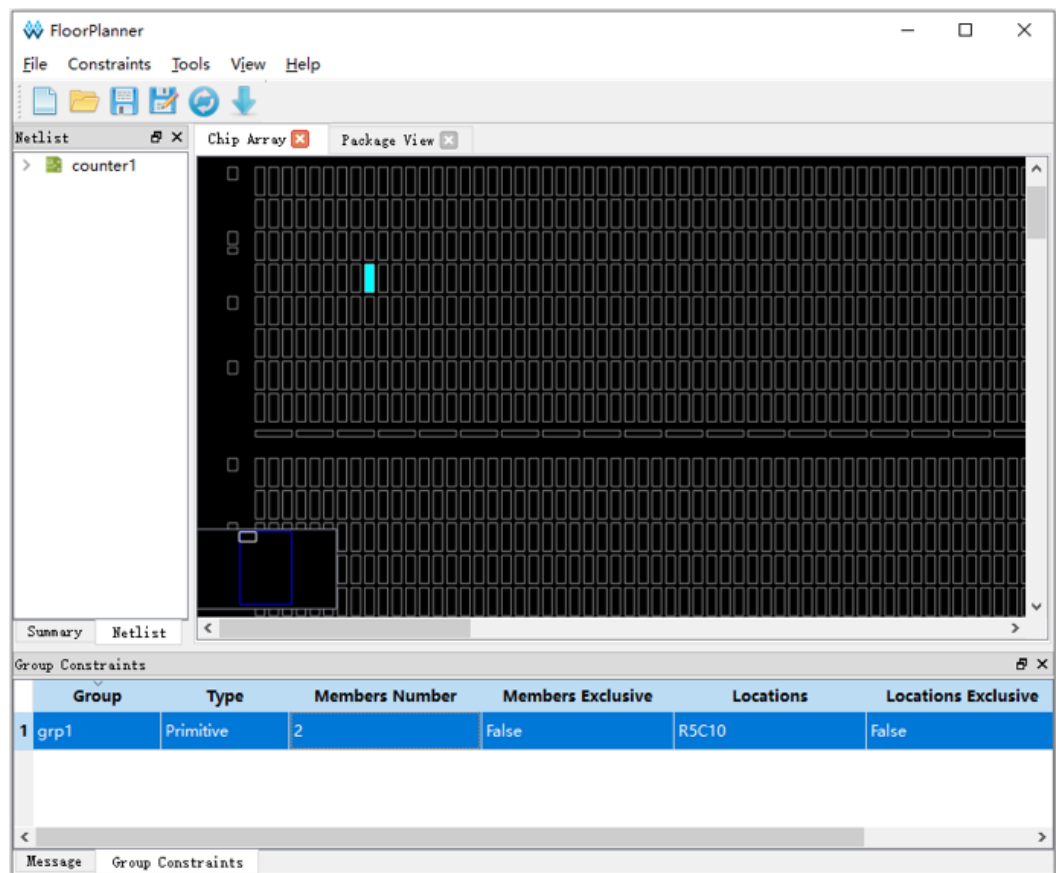


图 4-9 Primitive Group Constraints



注！

Primitive Group Constraints 的 Location 信息只能通过手动输入或者从 Chip Array 窗口中复制，不能通过拖拽实现。

编辑相对组约束

1. 在 Group Constraints 编辑窗口中，右击选择“New Relative Group”，弹出“New Relative Group”对话框；
2. 输入 Group 的名字“rel_grp”，单击“”，弹出 Select Primitives 对话框；
3. 在 Select Primitives 对话框中选择所要设置的 Primitives “cout_0_s1”、“cout_1_s1”，选择“OK”，添至“Member”列表中；
4. 为每个 Primitive 添加相对位置“R0C0”、“R4C5”，如图 4-10 所示；
5. 在 New Relative Group 对话框中，选择“OK”创建 Relative Group Constraints，如图 4-11 所示。

图 4-10 Relative Group Constraints 创建

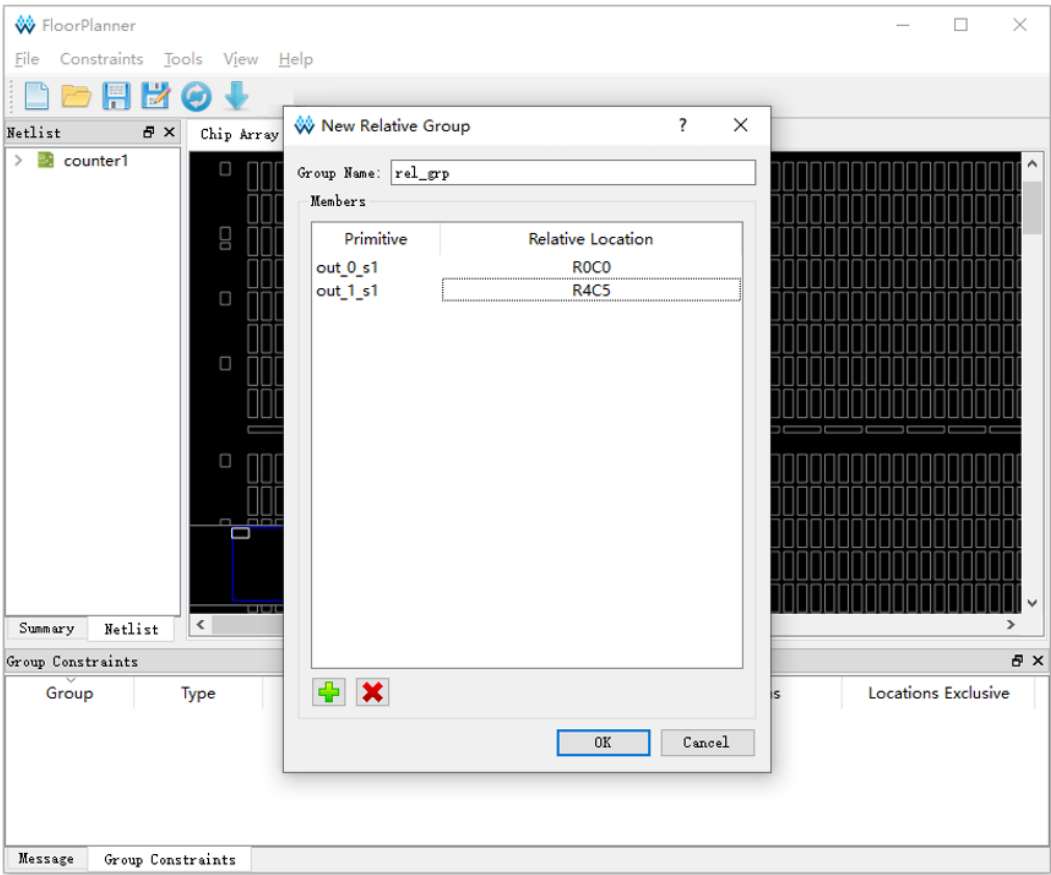
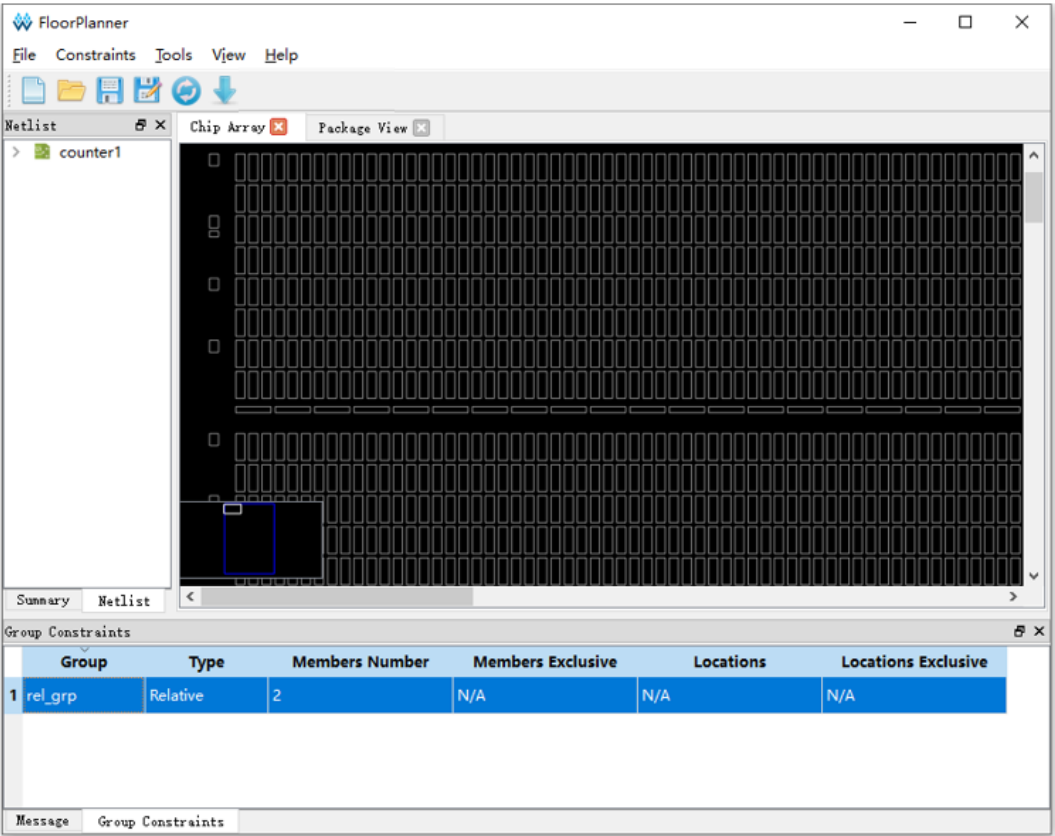


图 4-11 Relative Group Constraints



4.2.5 编辑资源预留约束

- 1. 在 Resource Reservation 编辑窗口中，进行右击选择 “Reserve Resources”，在编辑器中添加 Resource Reservation 约束，如图 4-12 所示；
- 2. 选中新建的 Resource Reservation 约束拖拽到 Chip Array 窗口中想要进行预留约束的位置，图 4-13 所示拖拽至 BSRAM_R10[1]位置完成 Resource Reservation 约束。

图 4-12 创建 Resource Reservation 约束

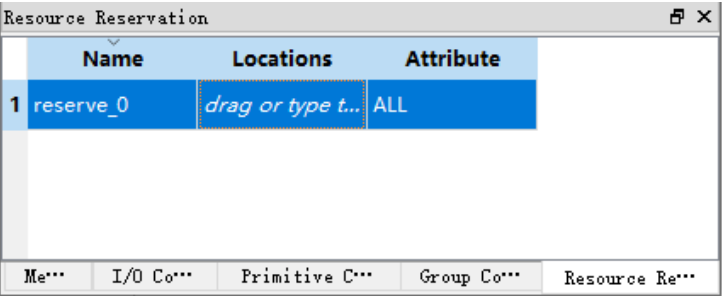
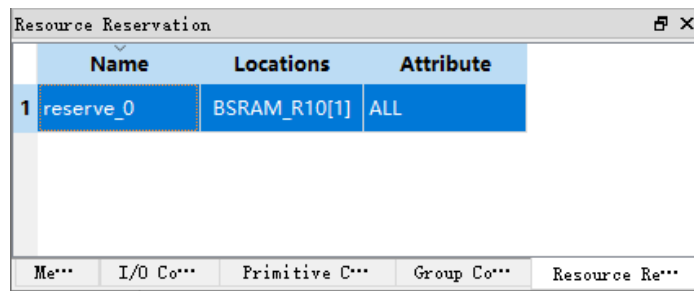


图 4-13 Resource Reservation



4.2.6 编辑全局时钟分配约束

1. 在 Clock Net Constraints 编辑窗口中，右击选择“Clock Net Constraints”，弹出“Clock Net Constraints”设置对话框；
2. 单击“”，弹出“Select Net”对话框，选择需要约束的 Net，在 Select Net 对话框中，单击“OK”添加 Net；
3. 设置时钟类型，在 Type 下拉列表中选择 Type 类型，设置 Signal 类型，如图 4-14 所示；
4. 设置完成后，单击“OK”，即会将该条约束添加至 Clock Net Constraints 编辑窗口中，如图 4-15 所示。

图 4-14 Clock Net Constraints 约束创建

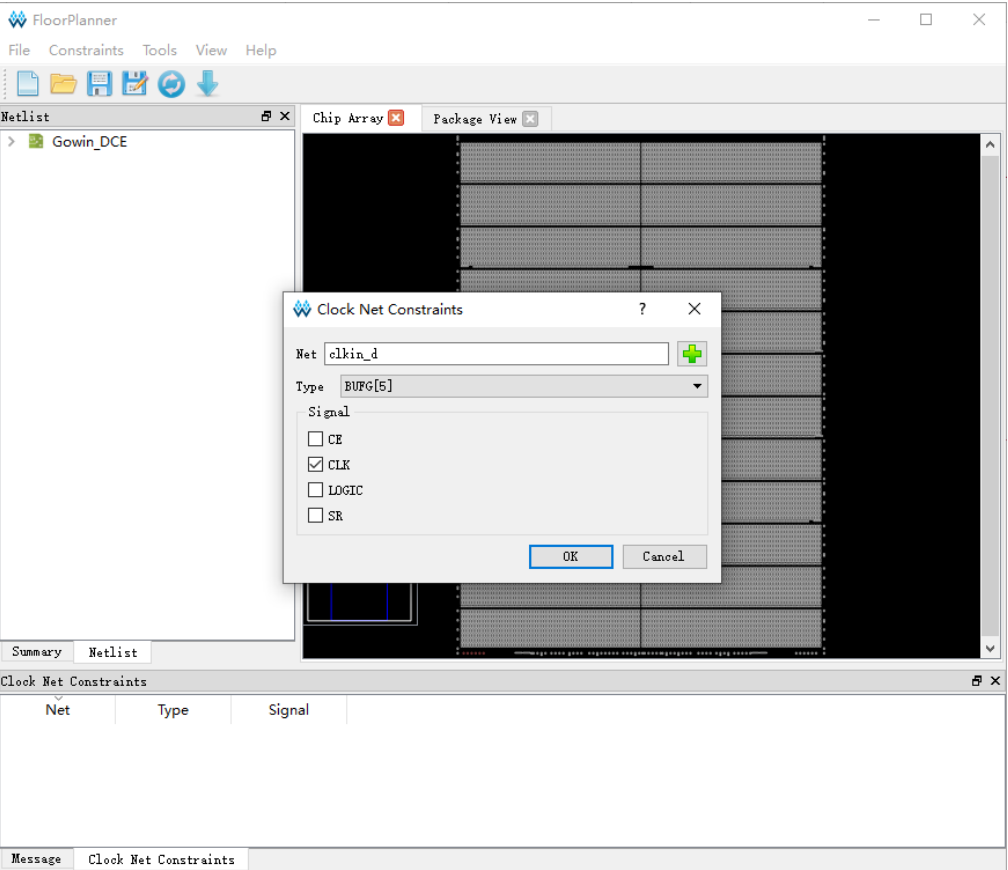
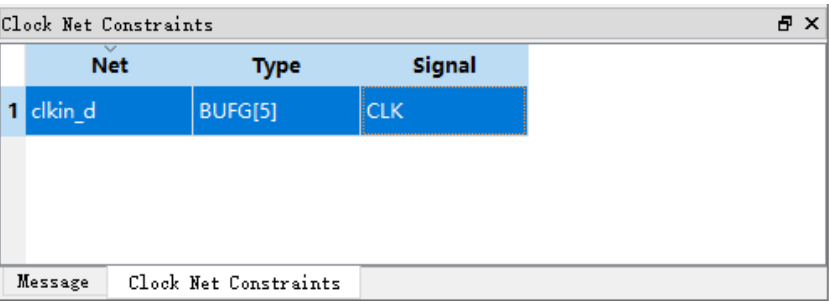


图 4-15 Clock Net Constraints 约束



4.2.7 编辑全局时钟约束

GCLK Primitive Constraints 仅支持对 DCS 和 DCE 的约束。

GCLK Primitive Constraints 的创建步骤如下：

- 1. 在 GCLK Primitive Constraints 编辑窗口中，进行右击选择“Select GCLK Primitive”操作，弹出“GCLK Primitive Constraints”对话框；
- 2. 单击 “”，弹出 GCLK Primitive 选择对话框，选取 Instance，在 GCLK Primitive 选择对话框中，单击 “OK”，Instance 完成设置；

3. 在 GCLK Primitive Constraints 对话框的“Position”下方选取所要约束的全局时钟位置，如图 4-16 所示；
4. 在 GCLK Primitive Constraints 对话框中单击“OK”，即将该约束添至 GCLK Primitive Constraints 编辑窗口，如图 4-17 所示。

图 4-16 GCLK Primitive Constraints 创建

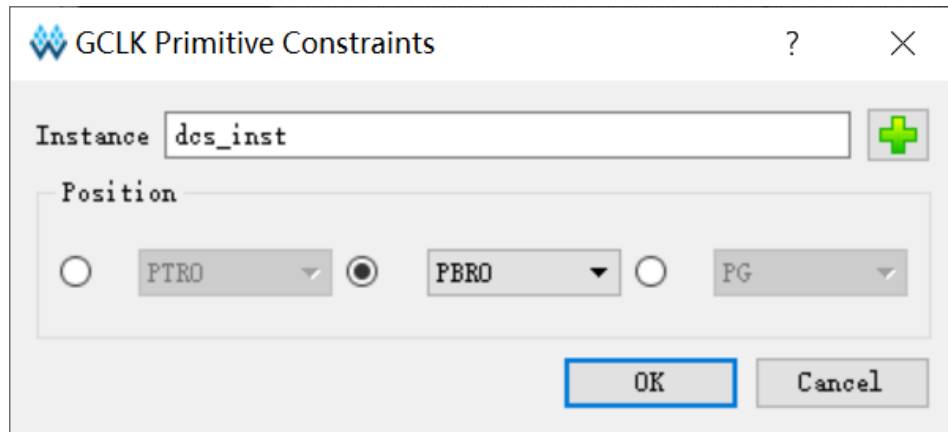
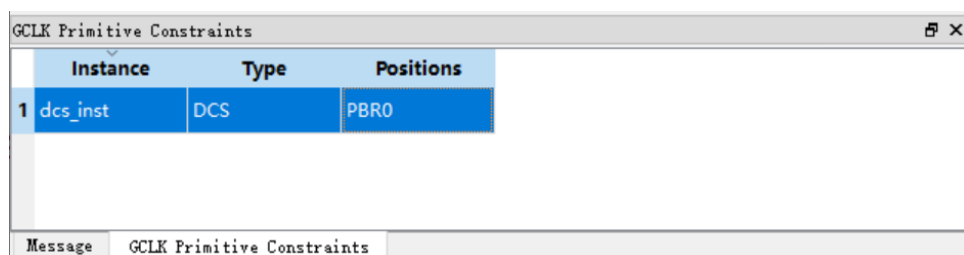


图 4-17 GCLK Primitive Constraints



4.2.8 编辑高速时钟约束

HCLK Primitive Constraints 仅对以下两种类型的 CLKDIV 和 DLLDL Instance 进行约束。

HCLK Primitive Constraints 的创建步骤如下：

1. 在 HCLK Primitive Constraints 编辑窗口中，进行右击选择“Select HCLK Primitive”，弹出“HCLK Primitive Constraints”对话框；
2. 单击“”按钮，弹出 HCLK Primitive 对话框，选取 Instance，在 HCLK Primitive 对话框中，单击“OK”，设置 Instance；
3. 在 HCLK Primitive Constraints 对话框中“Position”下方选取所要约束的高速时钟位置，如图 4-18 所示；
4. 单击 HCLK Primitive Constraints 对话框的“OK”，即可将该约束添至 HCLK Primitive Constraints 编辑器中，如图 4-19 所示。

图 4-18 HCLK Primitive Constraints 创建

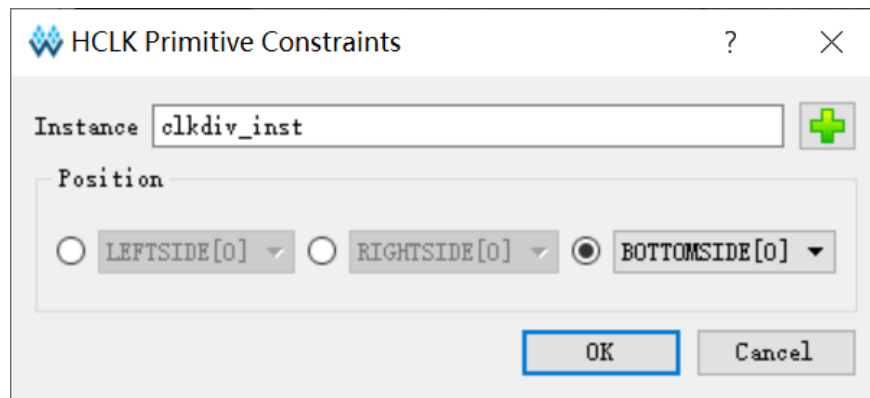
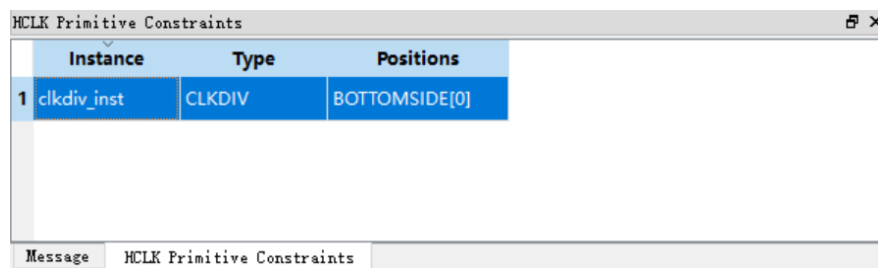


图 4-19 HCLK Primitive Constraints

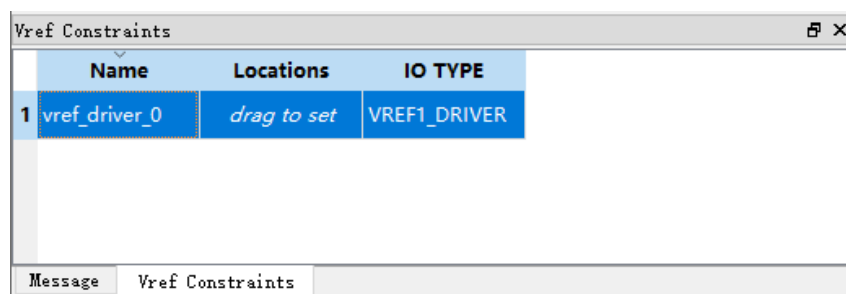


4.2.9 编辑参考电压约束

拖至 Chip Array 窗口创建 Vref Constraints，步骤如下：

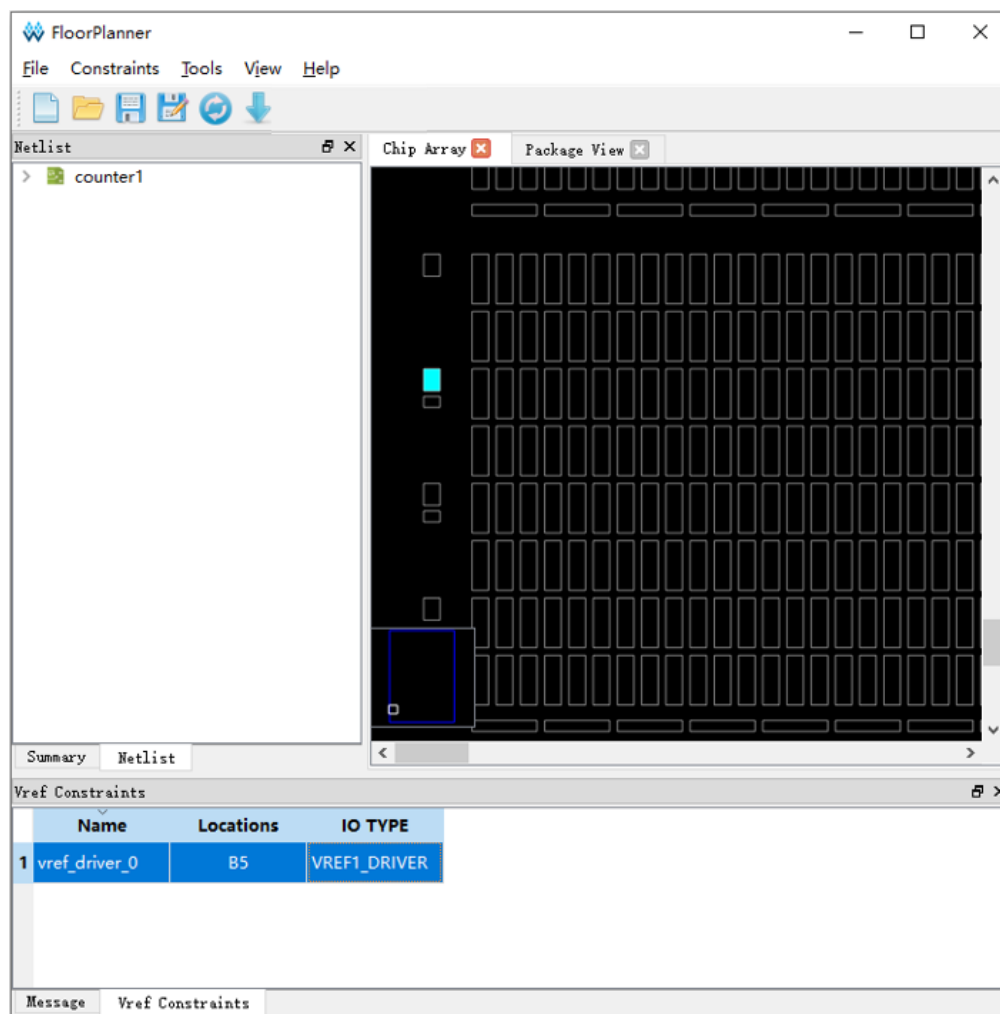
1. 在 Vref Constraints 编辑窗口中，进行右击选择“Define Vref Driver”，即可将该条 Vref Constraints 约束添至 Vref Constraints 编辑器中，如图 4-20 所示；
2. 放大 Chip Array 视图至宏单元模式，选中 Vref Constraints 编辑窗口中新创建的 Vref Constraints，拖拽至 Chip Array 窗口中的“B5”位置，Vref Constraints 的 Location 信息显示为“B5”，如图 4-21 所示。

图 4-20 Vref Constraints 创建



可自定义 Vref 约束名，Vref 名字不允许重名，设置期间，系统会进行检查，如名字重复，则提示用户，如图 4-23 所示。

图 4-21 拖拽至 Chip Array 窗口生成 Vref Constraints Location 信息



拖至 Package View 创建 Vref Constraints，步骤如下：

1. 在 Vref Constraints 编辑窗口中，进行右击选择“Define Vref Driver”，即可将该条 Vref Constraints 约束添至 Vref Constraints 编辑器中，如图 4-20 所示；
2. 选中 Vref Constraints 编辑窗口中新创建的 Vref Constraints，拖拽至 Package View 窗口中的“B5”位置，Vref Constraints 的 Location 信息显示为“B5”，如图 4-22 所示。

图 4-22 拖拽至 Package View 窗口生成 Vref Constraints location 信息

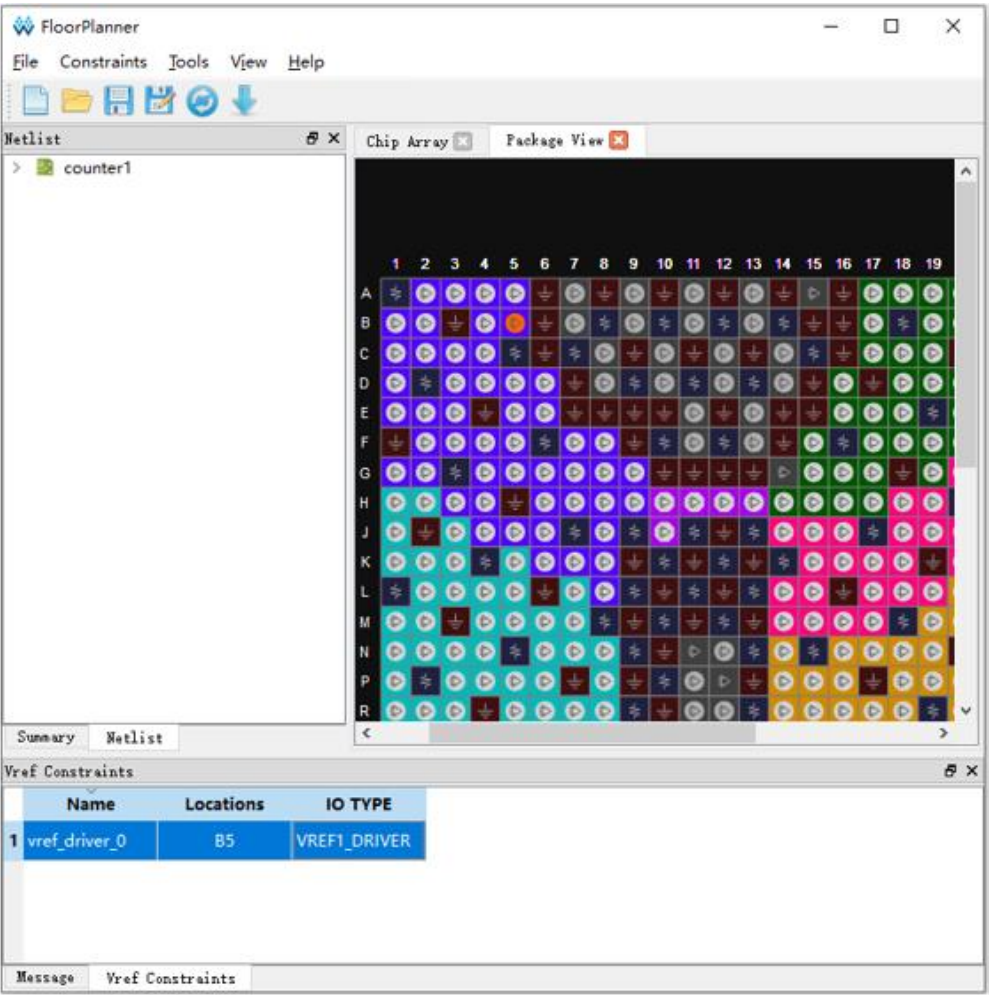


图 4-23 Vref Constraints 名字重复

